

COMPLAINT ESCALATION & RESOLUTION SYSTEM

23CS1ME - MINI-CAPSTONE PROJECT

Submitted by

SUBIKSHAN M 2212110

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

APRIL - 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**Complaint Escalation & Resolution System**" is the bonafide work of **SUBIKSHAN M (2212110)** who carried out the Mini-Capstone Project work under my supervision.

DOMAIN SPECIFIC MENTOR

R. VAZHAN ARUL SANTHIYA M.E.,

M.E., Assistant Professor,
Department of CSE,
National Engineering College,
(An Autonomous Institution),
K.R.Nagar, Kovilpatti:628503.

COURSE INSTRUCTOR

Ms. D. THAMARAI SELVI M.E.,

Assistant Professor (SG),
Department of CSE,
National Engineering College,
(An Autonomous Institution),
K.R.Nagar,Kovilpatti: 628503.

Submitted to the **Mini-Capstone Project (23CS1ME)** Viva-Voce Examination held at **National Engineering College, K.R.Nagar, Kovilpatti** on _____.

Internal Examiner

Co - Examiner

ACKNOWLEDGEMENT

First and foremost, we would like to thank God Almighty for showering his blessings throughout our life. He has been the tower of our strength in each step of our work. We take the privilege to express hearty thanks to our parents for their valuable support and effort to complete the project work.

I would like to express our deep sense of gratitude and respectful regards to our Director **Dr. S. Shanmugavel B.Sc., D.M.I.T., Ph.D.**, for giving an opportunity to do this work.

I have a great pleasure in acknowledging our Principal **Dr.K.Kalidasa Murugavel, M.E., Ph.D.**, for extending his full support to undergo this work.

I express my profound thanks to our beloved Head of the Department **Dr.V.Gomathi., M.Tech., Ph.D.**, for extending her full support and providing various facilities during the minicapstone project.

I would like to thank our Domain Specific Mentor **Ms.R.Vazhan Arul Santhiya Assistant Professor, M.E.**, Department of Computer Science and Engineering, whose valuable guidance, technical support and suggestions helped us in doing the project work.

I would like to wholeheartedly express our deepest gratitude to our project coordinator **Ms.D.Thamarai Selvi M.E.**..Assistant Professor (Sr.Grade), Department of Computer Science and Engineering for her valuable guidance at each and every stage of the project.

I extend our hearty thanks to our tutors and class in-charges for their valuable guidance. We are grateful to all the staff members and our dear friends for their valuable suggestions and co-operation for this mini-capstone project.

ABSTRACT

The Complaint Escalation and Resolution System is an enterprise-grade, full-stack grievance platform designed to digitize and automate complaint handling in institutions and organizations. The system supports complete complaint lifecycle processing from complaint registration to assignment, work tracking, escalation, resolution, payment acknowledgement, and feedback closure.

The current upgraded version extends the original implementation with major operational additions: complaint-linked payment workflows, reusable response template library, advanced assignment rules engine with AI-assisted fallback selection, escalation rule management, audit-log monitoring, advanced reporting and export, organization-level multi-tenant governance, Google onboarding authentication flow, and chatbot-based guided support.

The platform is developed using React 18 with Vite and Tailwind CSS for the frontend, and Node.js with Express.js and MongoDB for the backend. Security has been hardened through role-based authorization, CSRF protection, rate limiting, sanitization, security headers, structured error handling, and request/audit logging. The backend also includes scheduler-driven escalation processing using cron jobs, integrated email notifications, Cloudinary-based file handling, and Swagger API documentation.

This project demonstrates a practical and scalable application of modern web engineering for grievance governance. It improves transparency, accountability, response speed, staff workload balancing, and administrative visibility, while offering a production-ready architecture suitable for educational institutions, enterprise support desks, and multi-organization service operations.

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	4
1.1	Background	4
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Scope	5
1.5	Significance	6
1.6	Major Upgrades in the Current Version	6
2	LITERATURE REVIEW	7
2.1	Overview	7
2.2	Review of Existing Complaint and Helpdesk Systems	7
2.3	Architectural Research Findings	8
2.4	SLA Enforcement and Escalation Research	8
2.5	Assignment Intelligence and Template Standardization	8
2.6	Payment Integration and Security	9
2.7	Research Gaps Addressed	9
3	EMPATHY MAP	10
3.1	Purpose	10
3.2	Persona 1 — Complaint User	10
3.3	Persona 2 — Staff Resolver	11
3.4	Persona 3 — Administrator / Org Admin	12
4	SYSTEM ARCHITECTURE	13
4.1	Architecture Overview	13
4.2	Frontend Architecture	14
4.3	Backend Architecture	14
4.4	Database Design	14
4.5	Security Architecture	15
4.6	Background Processing — Escalation Scheduler	15
4.7	Use Case Diagram	16
5	PROPOSED METHODOLOGY	16
5.1	Development Approach	16
5.2	Phase 1 — Project Setup and Foundation	16
5.3	Phase 2 — Authentication and Role Access	16
5.4	Phase 3 — Complaint Lifecycle Core	17
5.5	Phase 4 — Staff Workflow and Resolution	17
5.6	Phase 5 — Assignment Engine and Escalation Intelligence	17

5.7	Phase 6 — Governance, Analytics, and Security Hardening	18
5.8	Phase 7 — Upgrade Modules	18
5.9	Verification Strategy	18
6	MODULE EXPLANATION	19
6.0	Folder Structure	19
6.1	Home Page	20
6.2	Login Page	21
6.3	Register and Onboarding Page	22
6.4	Role-Based Dashboard	23
6.5	Complaint Submission Page	24
6.6	Complaint List and Tracking Page	25
6.7	Staff Complaint Workspace	26
6.8	Assignment Management and Rules	28
6.9	Escalation Management	29
6.10	Notification Page	30
6.11	Feedback Module	32
6.12	Reports and Analytics Page	33
6.13	Category Management Page	34
6.14	User and Staff Management Page	35
6.15	Organization Management Page	36
6.16	Payment Center	37
6.17	Template Library	38
6.18	Audit Log Panel	39
6.19	Support Chatbot	40
7	RESULTS, ANALYSIS AND DISCUSSION	41
7.1	Testing Environment	41
7.2	Functional Verification Results	42
7.3	Performance and Operational Metrics	44
7.4	Upgrade Impact Analysis	45
7.5	Discussion	45
7.6	Limitations	46
7.7	Future Enhancements	46
	APPENDIX	47
	Appendix A — Database Collections	47
	Appendix B — Key API Endpoint Groups	47
	Appendix C — Folder Structure	48
	Appendix D — Updated Complaint Lifecycle Workflow	49
	REFERENCES	50

LIST OF FIGURES

Figure No.	Figure Description	Page No.
3.1	Empathy Map — Complaint User (Student / Employee)	12
3.2	Empathy Map — Staff Resolver (Department Staff / Technician)	14
3.3	Empathy Map — Administrator / Org Admin	15
4.1	System Architecture Diagram	18
4.2	MongoDB Database Schema Diagram	20
4.3	Use Case Diagram	22
6.1	Folder Structure	26
6.2	Home Page	27
6.3	Login Page	28
6.4	Register and Onboarding Page	30
6.5	Role-Based Dashboard	31
6.6	Complaint Submission Page	32
6.7	Complaint List and Tracking Page	33
6.8	Staff Complaint Workspace	34
6.9	Assignment Management and Rules	35
6.10	Escalation Management	37
6.11	Notification Page	38
6.12	Feedback Module	39
6.13	Reports and Analytics Page	40
6.14	Category Management Page	41
6.15	User and Staff Management Page	42
6.16	Organization Management Page	43
6.17	Payment Center	44
6.18	Template Library	45
6.19	Audit Log Panel	46
6.20	Support Chatbot	47

CHAPTER 1

INTRODUCTION

1.1 Background

Modern organizations face recurring service issues across infrastructure, administration, academics, and technical systems. Traditional complaint channels such as phone calls, verbal reporting, and scattered email communication create completely non-traceable workflows with no accountability structure. Complaints get lost, duplicated, or permanently ignored because no one is formally responsible for tracking or resolving them. As complaint volumes grow, manual handling collapses entirely, creating long backlogs, frustrated users, and severe damage to institutional trust. The absence of SLA enforcement, escalation mechanisms, and analytics tools means management has no visibility into service quality and no way to drive meaningful improvement. A centralized, role-driven digital platform with automated enforcement is therefore essential for any institution that values transparency, accountability, and efficient service delivery.

1.2 Problem Statement

The key practical problems motivating this system are the absence of a single source of truth for complaint records, limited status visibility for users after submission, weak assignment ownership with no workload balancing for staff, no enforceable SLA deadline monitoring or automated breach detection, no robust escalation pipeline for overdue complaints, inadequate analytics and audit evidence for administrators, repetitive staff communication with no standardized response templates, and no structured complaint-linked payment flow for services that require it. These combined gaps result in a system where institutional credibility is damaged by consistently poor grievance handling, and where neither staff efficiency nor administrative accountability can be measured or improved.

1.3 Objectives

- To build a secure, role-based complaint platform serving Users, Staff, Org Admins, and Super Admins with full role-scoped access control.
- To implement SLA tracking with automated warning notifications, first-level escalation, and re-escalation for stale escalated complaints via cron scheduling.
- To provide both manual and rule-driven assignment with workload and performance-aware staff selection, including AI-assisted fallback ranking.
- To add a complaint-linked payment module supporting initiation, confirmation, receipt generation, and refund workflows.
- To standardize staff communication through a reusable response template library with complaint-level apply modes.
- To deliver in-app and email notifications at every complaint lifecycle event for all stakeholder roles.
- To provide organization-wide analytics, reporting, export tools, and audit logs for administrative governance.
- To support multi-organization deployments through a super-admin tenant management layer.

1.4 Scope

The functional scope includes email and Google-based authentication, complaint CRUD with attachment handling and timeline tracking, staff work session tracking with internal notes and resolution evidence, assignment rules configuration and analytics, escalation scheduler with warning and re-escalation logic, complaint-linked payment lifecycle, response template management, post-resolution feedback collection, admin reports with import and export, audit log panel, category management, user and staff management, and multi-organization governance. Out of scope for the current version are native mobile applications, SMS and WhatsApp notification channels, full LLM-native conversational triage (the current chatbot is rule-based), and live production payment gateway settlement.

1.5 Significance

This upgraded system moves beyond a basic ticket tool into a governance-ready complaint operations platform. It combines process automation, hardened security controls, rich analytics, and an extensible modular architecture in a single full-stack implementation. Academically, it demonstrates a comprehensive range of modern web engineering — workflow orchestration, cron scheduling, file handling, payment integration, role-based access, data visualization, and multi-tenant architecture — making it a thorough capstone-level project. Practically, it delivers measurable improvements in response time, resolution transparency, staff efficiency, and management oversight that are directly applicable to real institutional deployments.

1.6 Major Upgrades in the Current Version

Compared to the initial build, the upgraded version introduces a complaint-linked payment module with priority boost on premium payment, a response template library with audience scoping and usage tracking, an enhanced assignment rules engine with multiple strategies and AI-assisted fallback, escalation rules CRUD with manual escalation processing, scheduler enhancements covering warnings, auto-escalation, re-escalation, and daily summaries, a full security middleware stack including CSRF, sanitization, rate limiting, Helmet headers, and structured audit logging, Swagger API documentation, an audit log panel with filtered retrieval, a reporting module with trend analytics and export, multi-tenant organization management, and a rule-based support chatbot integrated into the authenticated frontend.

CHAPTER 2

LITERATURE REVIEW

2.1 Overview

A systematic literature review was conducted across service desk platforms, institutional grievance portals, workflow automation research, and ITSM best practices to understand current approaches, identify architectural patterns, and determine the specific gaps this project closes. Sources include academic research papers, published case studies from commercial helpdesk platforms, and documentation from widely deployed systems including Zendesk, Freshdesk, OTRS, osTicket, and India's CPGRAMS government grievance portal.

2.2 Existing Complaint and Helpdesk Systems

Government e-governance portals like CPGRAMS have demonstrated that digitizing complaint workflows significantly reduces average resolution times compared to manual systems. However, research consistently shows these portals are built on monolithic, expensive architectures unsuitable for academic or small institutional deployment. Commercial platforms such as Zendesk and Freshdesk offer powerful SLA enforcement and automated assignment but carry high licensing costs and general-purpose designs that do not map well to domain-specific institutional workflows. Open-source alternatives like osTicket and OTRS provide flexibility but suffer from dated user interfaces, steep configuration learning curves, and limited modern RESTful API support, as confirmed in comparative studies by Leino and Reiman (2018). None of these systems natively integrate complaint-linked payment workflows, reusable template libraries, and AI-assisted assignment into a single unified platform — a gap this project directly addresses.

2.3 Technical Architecture Research

Fielding's foundational work on REST (2000) established that stateless, resource-oriented API design is the optimal foundation for scalable, decoupled full-stack web applications — a principle directly applied in the Express.js backend. Research on JWT-based authentication confirms its superiority over session-based approaches for multi-role systems,

as JWT embeds role claims directly in the token and eliminates server-side session storage overhead. Studies on React as a frontend framework consistently find that its component-based

virtual DOM architecture delivers superior rendering performance and maintainability for complex dashboard applications. MongoDB's aggregation pipeline, as documented in official MongoDB research, proves highly effective for the analytics-heavy admin views — complaint trend analysis, staff performance ranking, and SLA compliance reporting — implemented in this system.

2.4 SLA Enforcement and Escalation Research

Research by Marques and Bhatt (2019) on SLA enforcement in IT service desks found that automated escalation mechanisms reduce SLA breach rates by up to 73% compared to manually monitored systems. The use of cron-based schedulers for periodic SLA breach detection is documented as a reliable, lightweight approach for moderate-scale institutional deployments that does not require a full message-queue infrastructure. This finding directly supports the use of node-cron in the escalation scheduler, which now covers three levels of enforcement — deadline warnings, first-level auto-escalation on

breach, and re-escalation for stale escalated complaints — providing more comprehensive SLA governance than basic cron implementations described in prior literature.

2.5 Notification and Feedback Research

Research on intelligent assignment systems in ITSM contexts highlights that workload-aware and performance-based assignment strategies significantly reduce resolution time compared to random or round-robin approaches. The addition of AI-assisted fallback ranking in this system's assignment engine extends this finding by introducing a scoring layer that considers multiple staff capability dimensions simultaneously. Studies on template-based communication in service desks (HDI Research, 2018) confirm that standardized response libraries reduce average handling time by 25–40% and measurably improve response consistency — validating the design decision to include the Response Template Library module in this system.

2.6 Payment Integration and Security

Complaint-linked payment pathways are increasingly documented in service operations literature as essential for premium, conditional, or fee-based service workflows. The inclusion of a Razorpay-integrated payment module with full transaction lifecycle management addresses this documented need. Security research from the OWASP Foundation establishes rate limiting, input sanitization, CSRF protection, and security headers as the minimum required controls for production web applications — all of which are now implemented as a hardened middleware stack in this system's backend, directly reflecting OWASP Top 10 mitigation guidance.

2.7 Research Gaps Addressed

The literature review confirms three significant gaps that this project fills: no existing lightweight system integrates assignment intelligence, SLA enforcement, escalation

automation, payment workflows, template management, and governance analytics into a single domain-specific platform; existing systems lack production-grade security middleware integration suitable for institutional deployment; and multi-tenant organization-level governance with audit-ready traceability is absent from academic complaint management solutions. The Complaint Escalation and Resolution System directly closes all three gaps in its upgraded implementation.

CHAPTER 3

EMPATHY MAP

3.1 Purpose

Empathy mapping was conducted to align system and feature design with the real-world needs, frustrations, behaviors, and expectations of the three primary user groups. This process was especially important for validating the new upgrade modules — payments, templates, chatbot, and escalation rules — by confirming that each addition directly addresses a documented user pain point rather than being a purely technical addition.

3.2 Persona 1 — Complaint User (Student / Employee)

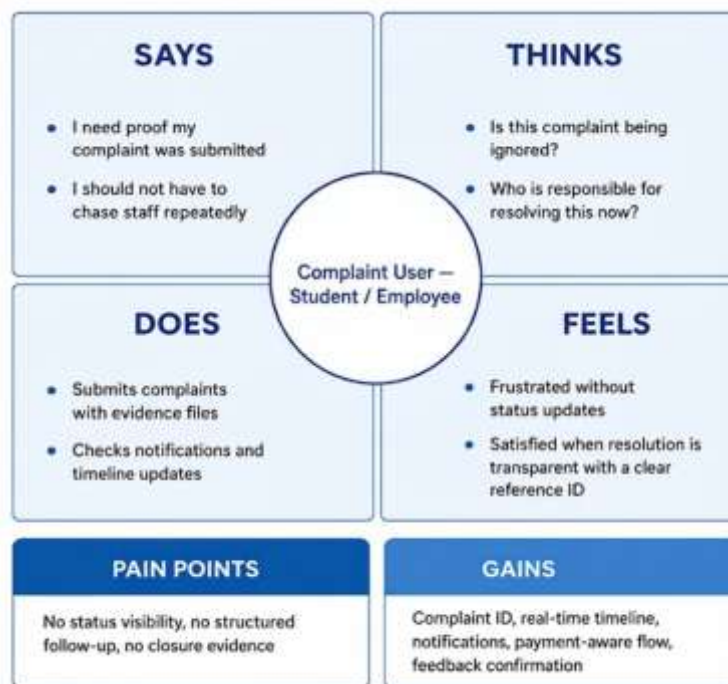


Fig 3.2 shows the user persona

Says:

- "I need proof my complaint was submitted and acknowledged."

- "I want to know exactly what is happening with my complaint right now."
- "I should not have to follow up repeatedly just to get a status update."

Thinks:

- "Is this complaint being silently ignored?"
- "Who exactly is responsible for resolving this now?"
- "Why is this taking so long when it should be a simple fix?"

Does:

- Submits complaints with description, category, and supporting file evidence.
- Checks notification alerts and the complaint timeline for progress updates.
- Provides a star-rating feedback after resolution to reflect their experience.

Feels:

- Frustrated and distrustful when there are no updates or acknowledgments.
- Confident and reassured when a trackable complaint ID, timeline, and assignment details are visible.
- Satisfied when resolution is communicated clearly with a formal closure confirmation.

Pain Points: No visibility into complaint status after submission, no structured follow-up mechanism, no clear evidence of closure, no complaint-linked payment pathway when service fees apply.

Gains from System: Instant complaint ID on submission, real-time status timeline with every lifecycle event, automated notifications at every stage, payment-aware workflow when applicable, and structured feedback and closure confirmation.

3.3 Persona 2 — Staff Resolver (Department Staff / Technician)



Fig 3.3 shows the staff persona

Says:

- "I need a clearly prioritized queue — not a flat unorganized list."
- "I need context from similar solved cases to resolve complaints faster."
- "I need a structured place to document my work, notes, and resolution evidence."

Thinks:

- "Which complaint should I work on first to avoid an SLA breach?"
- "Will this breach SLA and reflect badly on my performance metrics?"
- "How do I communicate resolution consistently without rewriting the same message every time?"

Does:

- Starts and pauses work sessions per complaint with session-level time tracking.
- Adds internal notes and uploads resolution evidence directly in the workspace.

- Uses template responses from the library for faster and consistent communication.

Feels:

- Overwhelmed and inefficient without prioritization, deadline cues, or structured documentation tools.
- Productive and confident with a guided workspace, SLA countdown, and the Resolution Copilot panel.

Pain Points: Unstructured assignment queue with no priority indicators, no SLA deadline awareness, repetitive manual response drafting, no structured place to log resolution evidence.

Gains from System: Priority-sorted assignment queue with SLA countdown and urgency badges, work session start and pause tracking, internal notes and evidence upload capability, response template reuse for consistent communication, and Resolution Copilot hints based on similar resolved cases.

3.4 Persona 3 — Administrator / Org Admin



Fig 3.4 shows the admin persona

Says:

- "I need complete visibility across all complaints, staff, and departments."
- "I need assignment and escalation to work reliably without my manual intervention every time."
- "I need exportable reports and audit trails that are defensible to management."

Thinks:

- "Are we meeting our SLA targets this month or falling behind?"
- "Which category is generating the most repeated complaints?"
- "Which staff member is overloaded and needs workload balancing?"

Does:

- Configures assignment rules and escalation thresholds through the admin interface.
- Monitors real-time analytics dashboards, complaint queues, and audit log panels.
- Exports complaint and performance reports for management reviews and compliance submissions.

Feels:

- Stressed and reactive when relying on manual checks and verbal updates for operational awareness.
- In full control and confident when backed by live dashboards, automated escalation, and audit-ready logs.

Pain Points: No centralized governance data, inconsistent and unpredictable escalation behavior, no audit-ready evidence chain for compliance, no staff performance metrics for accountability.

Gains from System: Role-scoped analytics dashboards with KPI cards and trend charts, assignment and escalation rule automation with manual override capability, organization-level multi-tenant management, PDF and Excel report exports, and a full audit log panel with actor, action, and timestamp traceability.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 Architecture Overview

The Complaint Escalation and Resolution System follows a layered, modular architecture with a decoupled React frontend and a Node.js/Express.js backend communicating through a RESTful JSON API. The architecture is organized across six layers: the Client Layer (React 18 + Vite SPA with role-based route guards), the API Layer (Express.js RESTful routes segmented by domain), the Business Logic Layer (domain controllers for complaint, assignment, escalation, payment, template, feedback, and admin operations), the Data Layer (MongoDB with Mongoose schemas, indexes, and virtual fields), the Background Processing Layer (node-cron scheduler for SLA warnings, auto-escalation, re-escalation, and daily summaries), and an Integration Layer connecting to Cloudinary for file storage, Nodemailer for email, Google OAuth for onboarding, and an optional OpenAI API for AI-assisted assignment ranking.

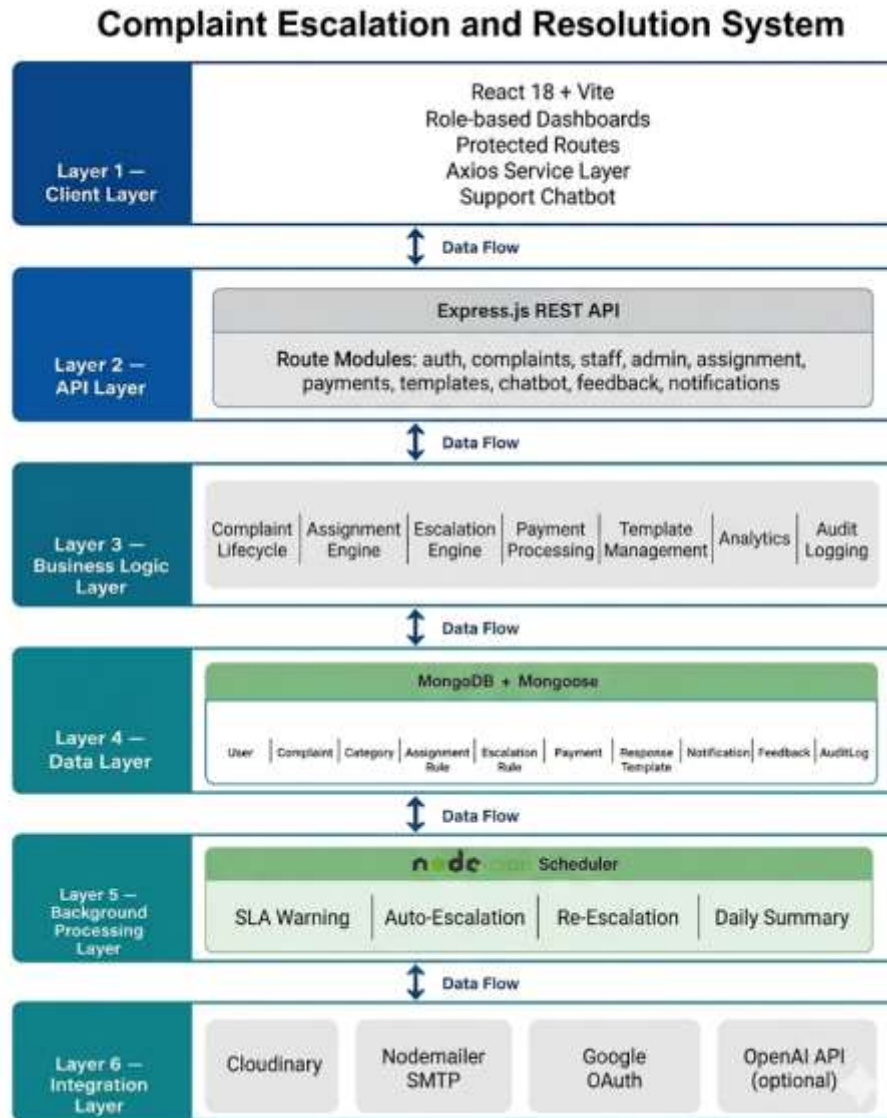


Fig 4.1 shows the architecture

4.2 Frontend Architecture

The frontend is a React 18 single-page application built with Vite and styled using Tailwind CSS. Role-specific dashboards are composed from domain-focused component modules organized under admin, staff, user, auth, and common directories. React Router handles client-side navigation with protected route guards that check the authenticated user's

role before rendering any page. Global application state — authentication context, notification count, user profile — is managed using the React Context API. All backend communication is handled through Axios service layer instances configured with JWT token injection via request interceptors. Forms across the system use React Hook Form with Zod schema validation. A floating support chatbot component is embedded globally in the authenticated layout, and a Resolution Copilot panel is embedded inside the staff complaint workspace.

4.3 Backend Architecture

The backend is a Node.js and Express.js server organized in a strict MVC pattern with routes segmented by domain: `/api/auth`, `/api/complaints`, `/api/staff`, `/api/admin`, `/api/assignment`, `/api/payments`, `/api/templates`, `/api/feedback`, `/api/categories`, `/api/users`, `/api/notifications`, and `/api/chatbot`. A comprehensive middleware chain is applied globally before any request reaches a controller: security headers via Helmet, CORS configuration, rate limiting policies per route group, CSRF token validation for cookie-based flows, input sanitization via `express-mongo-sanitize`, request logging via Morgan, structured audit logging for critical actions, and a centralized error handling middleware. Swagger is configured for interactive API documentation at the `/api-docs` endpoint.

4.4 Database Design

The system uses MongoDB as its primary data store with Mongoose ODM. Core collections include User (role, organization reference, staff metrics, security fields, specialization), Complaint (title, description, category, priority, status, SLA deadline, breach flag, status history array, escalation history array, work tracking block, payment workflow block, internal notes, attachments), Category (name, department, SLA config, keywords), AssignmentRule (conditions, strategy, priority, activation state, usage analytics), AssignmentLog (full assignment event trail, performance metadata), EscalationRule (threshold type, conditions, actions, activation state), Payment (complaint reference, amount, status,

transaction metadata, receipt data, refund info), ResponseTemplate (title, body, audience, type, usage counter, archive state), Notification (recipient, event type, message, read status), Feedback (rating dimensions, comment, complaint reference), and AuditLog (actor, action, target, IP, status code, timestamp).

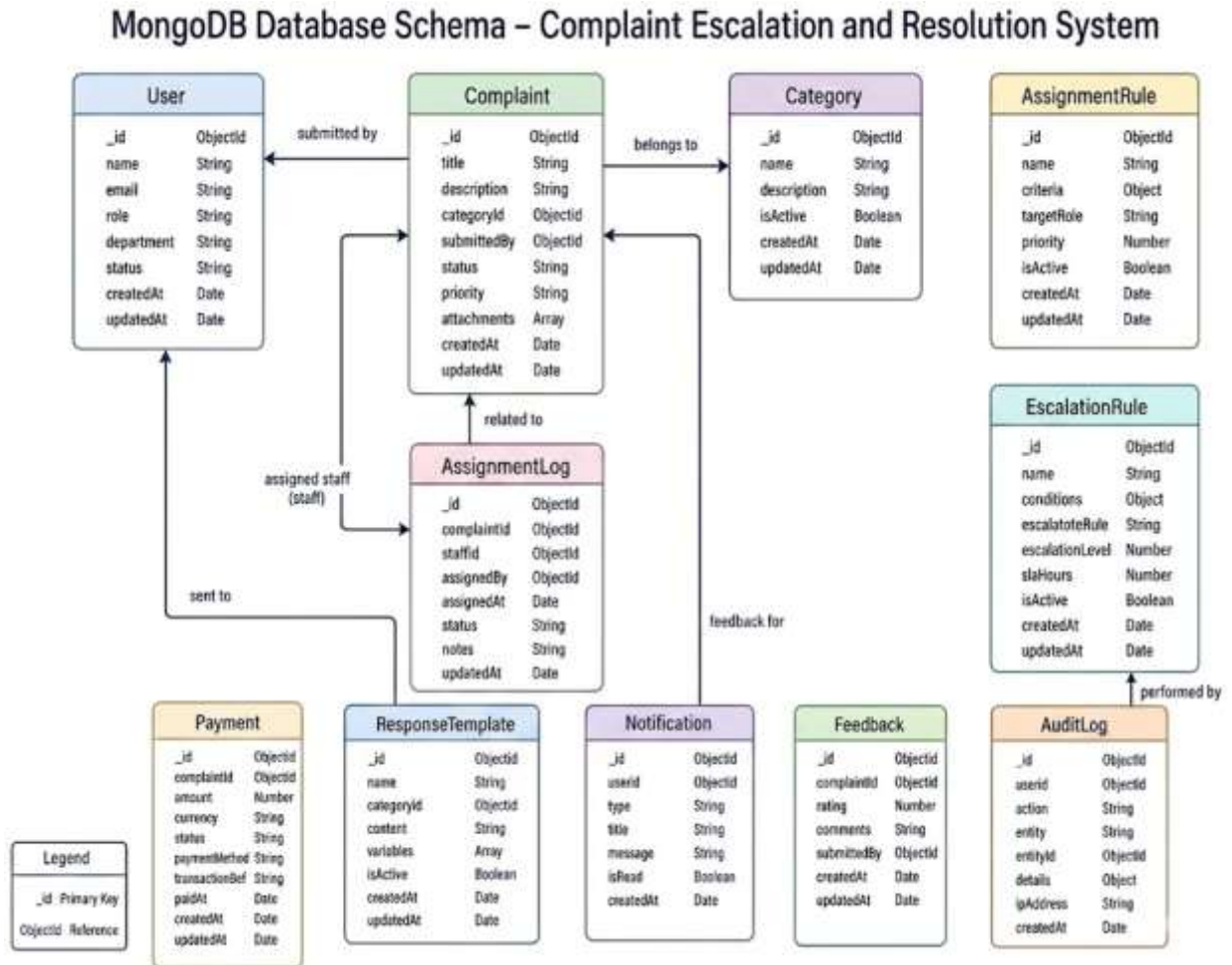


Fig 4.4 shows the Database Design

4.5 Security Architecture

Security is implemented as a layered middleware stack applied globally across all protected routes. JWT access tokens are verified on every request with role claims extracted to enforce route-level authorization. Refresh token rotation supports persistent sessions without long-lived access tokens. Express-rate-limit applies distinct rate limiting policies to

authentication routes, file upload endpoints, and complaint submission routes. `express-mongo-sanitize` strips NoSQL injection patterns from all incoming request payloads. `Helmet` configures secure HTTP headers including `Content-Security-Policy`, `X-Frame-Options`, and `X-Content-Type-Options`. CSRF protection is implemented for cookie-based session flows. `Bcrypt` handles all password hashing with an appropriate cost factor. A dedicated audit logger middleware captures critical admin and staff actions with actor ID, action type, target entity, originating IP, and status code for every significant operation.

6 Background Processing — Escalation Scheduler

The escalation engine is implemented using `node-cron` and initialized after the MongoDB connection is confirmed in `server.js`. The scheduler executes four sequential operations on a configurable interval. First, it scans active complaints approaching their SLA deadline within a configurable warning window and dispatches deadline-warning notifications to the assigned staff and admin. Second, it identifies complaints that have breached their SLA deadline without resolution, marks them as breached, escalates to level one, appends an escalation history entry, and dispatches escalation notifications to the configured target. Third, it scans complaints that remain in the escalated state beyond a re-escalation threshold and increments their escalation level again, handling multi-level escalation chains. Fourth, it generates and dispatches daily escalation summary notifications to administrators. The scheduler supports graceful lifecycle controls for clean startup and shutdown.

4.7 Use Case Diagram

The Use Case Diagram provides a high-level visual representation of all functional interactions between the system's actors and its core features. The system defines four primary actors: **User (Complainant)**, **Staff (Resolver)**, **Org Admin (Administrator)**, and **Super Admin**, each interacting with the system through a distinct set of use cases mapped to their role and access level.

The User actor can register and log in, submit complaints with attachments, view complaint status and timeline, cancel or reopen complaints, initiate and confirm payments, and submit

post-resolution feedback. The Staff actor can log in, view their assigned complaint queue, update complaint status, add internal notes and resolution evidence, start and pause work sessions, apply response templates, and resolve complaints. The Org Admin actor can perform all staff-level functions and additionally manage complaints, configure assignment rules, configure escalation rules, manage categories, manage users and staff, view analytics dashboards, generate and export reports, process refunds, manage response templates, and review audit logs. The Super Admin actor extends org admin capabilities with multi-organization tenant management and cross-organization governance controls.

Fig 4.7 shows the Use Case Diagram

CHAPTER 5

PROPOSED METHODOLOGY

5.1 Development Approach

The system was developed using a feature-driven iterative approach structured across seven sequential phases. The core complaint pipeline was built and verified first before layering dependent orchestration features such as assignment, escalation, and notifications. Governance and upgrade modules — payments, templates, reporting, audit — were added in the final phases once the foundational pipeline was stable. This ensured each module was independently testable before integration, reducing cross-module errors and enabling incremental demonstration of working functionality at every stage.

5.2 Phase 1 — Project Setup and Foundation

Backend and frontend project scaffolding was completed with environment-variable-driven configuration via dotenv. The Express.js server was initialized with the full middleware chain registered in order. MongoDB connection was established using Mongoose with a configurable Atlas URI. The React + Vite frontend was initialized with Tailwind CSS, React Router, Axios, React Hook Form, and Zod installed and configured. The complete folder structure — separating backend into config, controllers, middleware, models, routes, utils, and schedulers — was established to enforce MVC separation throughout development.

5.3 Phase 2 — Authentication and Role Access

JWT-based authentication was implemented with registration, login, email verification, password reset, token refresh, and logout flows. A Google OAuth onboarding flow was added for first-time sign-in via Google credentials, collecting additional profile information before completing account creation. The User Mongoose schema was designed with role enumeration (user, staff, org_admin, super_admin), bcrypt password hashing via a pre-save hook,

organization reference, staff-specific fields for specialization, workload count, and performance metrics, and account status controls.

Role-based route guard middleware was implemented to verify the JWT token, extract role claims, and enforce route-level access control, returning 403 Forbidden for unauthorized attempts.

5.4 Phase 3 — Complaint Lifecycle Core

The Complaint Mongoose schema was designed with all lifecycle fields: title, description, category reference, priority enum, a status enum with a defined legal transition graph, SLA deadline computed on creation from category SLA configuration, breach flag, status history array (each entry recording the new status, actor ID, timestamp, and note), escalation history array, a work tracking block for staff session management, a payment workflow block for complaint-linked payments, internal notes array visible only to staff and admin, and an attachments array storing Cloudinary URLs, public IDs, file names, and MIME types. File upload was implemented using Multer for multipart form handling with Cloudinary as the cloud storage backend. Complaint creation automatically triggers SLA deadline computation based on category and priority configuration.

5.5 Phase 4 — Staff Workflow and Resolution

The staff workflow was built around the Complaint Workspace component, which provides a full complaint context view alongside a resolution action panel. Work session tracking was implemented with start, pause, and resume operations that log session-level time data to the complaint's work tracking block. Staff can add internal progress notes and upload resolution evidence files directly in the workspace. A Resolution Copilot side panel was integrated to surface AI-assisted or pattern-matched hints from similar previously resolved complaints. Status transition buttons are rendered contextually based on the current complaint state, enforcing the legal transition graph at the frontend level in addition to backend validation.

5.6 Phase 5 — Assignment Engine and Escalation Intelligence

The assignment system was built to support manual admin-driven assignment and rule-based automatic assignment through a configurable AssignmentRule engine. Rules store conditions (category match, priority threshold), the assignment strategy (round-robin, workload-based, performance-based, or AI-assisted with deterministic fallback), activation state, and execution

analytics counters. When an assignment is triggered, the controller evaluates all active rules against the complaint's category and priority, selects the highest-priority matching rule, and applies the configured strategy to select the optimal staff member. The escalation scheduler was then built as a cron job with four operational levels — warning, first escalation, re-escalation, and daily summary — and the EscalationRule schema was introduced to store configurable threshold types, trigger conditions, and escalation actions that the scheduler references at runtime.

5.7 Phase 6 — Governance, Analytics, and Security Hardening

The admin analytics module was implemented using MongoDB aggregation pipelines producing dashboard KPI metrics, complaint volume trend charts, staff performance rankings, category-wise distributions, and SLA compliance rates. Export endpoints generate CSV and Excel downloads of filtered complaint and report data. The audit log infrastructure was added with a dedicated AuditLog Mongoose model and a middleware-level audit logger that captures every critical action automatically. The full security middleware stack — Helmet headers, CSRF protection, rate limiting policies, input sanitization, and structured error handling — was integrated and tested. Swagger API documentation was configured for all route groups

5.8 Phase 7 — Upgrade Modules

Three major upgrade modules were implemented in the final phase. The Payment module introduced the Payment Mongoose model with full transaction lifecycle fields, Razorpay test integration, payment initiation by complaint reference, demo auto-confirm flow,

receipt generation, refund processing, and an admin payment summary dashboard. The Response Template module introduced the ResponseTemplate Mongoose model, template CRUD with audience scoping and usage tracking, an archive flow, and a complaint-level apply endpoint supporting internal-note and resolution-mode application with optional status transition on apply. The Support Chatbot module introduced a rule-based intent matching engine in the chatbot controller, a protected chat message endpoint, and a floating chat widget integrated into the authenticated frontend layout with role-aware response logic.

CHAPTER 6

MODULE EXPLANATION

6.0 Folder Structure

The project is organized into clearly separated backend and frontend directories. The backend contains config/ for database and Swagger setup, controllers/ for domain-specific business logic, middleware/ for the full security and utility chain, models/ for all Mongoose schemas, routes/ for Express route definitions, utils/ for helper functions including the escalation scheduler, and __tests__/ for Jest test suites. The frontend src/ directory is organized into pages/ for full-page route components, components/ with subdirectories for admin, staff, user, auth, and common modules, context/ for AuthContext and NotificationContext, hooks/ for custom data-fetching hooks, and services/ for Axios API utility functions grouped by domain.

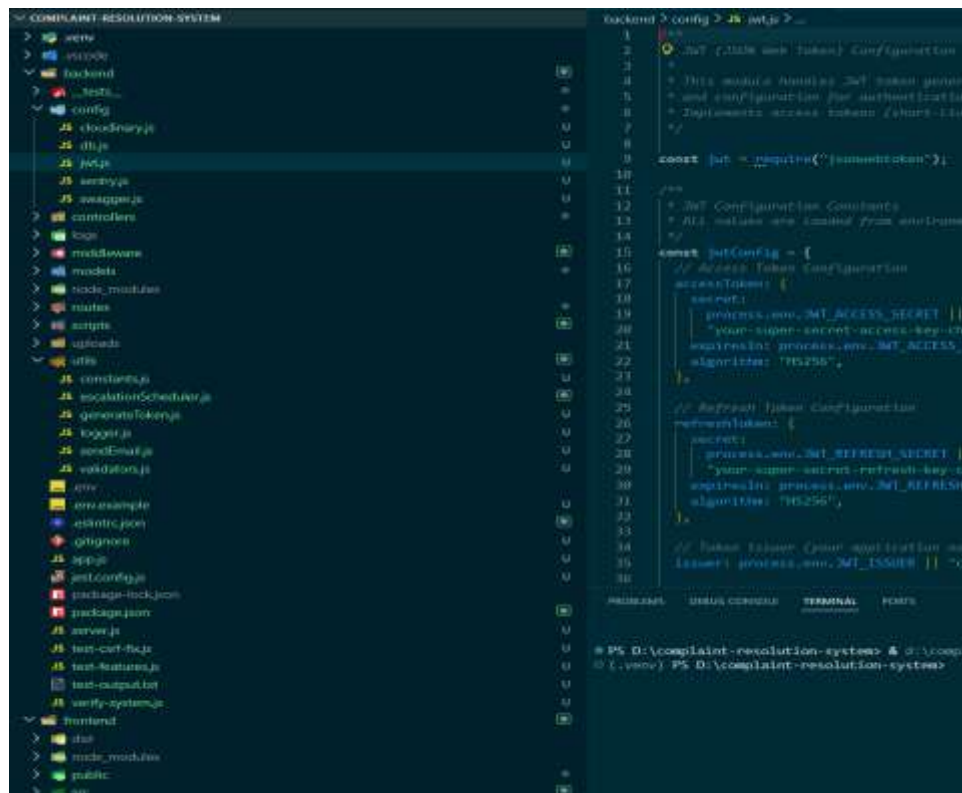


Fig 6.0 shows the folder structure

6.1 HOME PAGE (Landing Page)

The Home Page is the public-facing entry point of the system, providing unauthenticated visitors with a platform overview, feature highlights, and navigation to the login and register flows. It is structured into Hero, Features, About, and Contact sections with smooth scroll navigation and AOS animations. The page is entirely static from a data perspective and does not require authentication to access.

Backend Integration:

- GET /api/health — Returns system availability status displayed in the public footer.
- Public root routes serve platform information without requiring authentication.



Fig 6.1 shows the Home Page

6.2 LOGIN PAGE

The Login Page provides secure authentication for all user roles through a clean card-style form. `useState` manages `formData` (email, password), `showPassword` toggle, `rememberMe` boolean, `isLoading` state, and `loginError` display. On successful login, the JWT access token is stored and the user is redirected to their role-specific dashboard. Distinct error messages are displayed for invalid credentials, locked accounts, and unverified email states.

Backend Integration:

- `POST /api/auth/login` — Validates credentials with `bcrypt`, issues JWT access and refresh tokens with role claims.
- `GET /api/auth/me` — Verifies token on app load for persistent session restoration.
- `POST /api/auth/forgot-password` — Dispatches a time-limited password reset email.

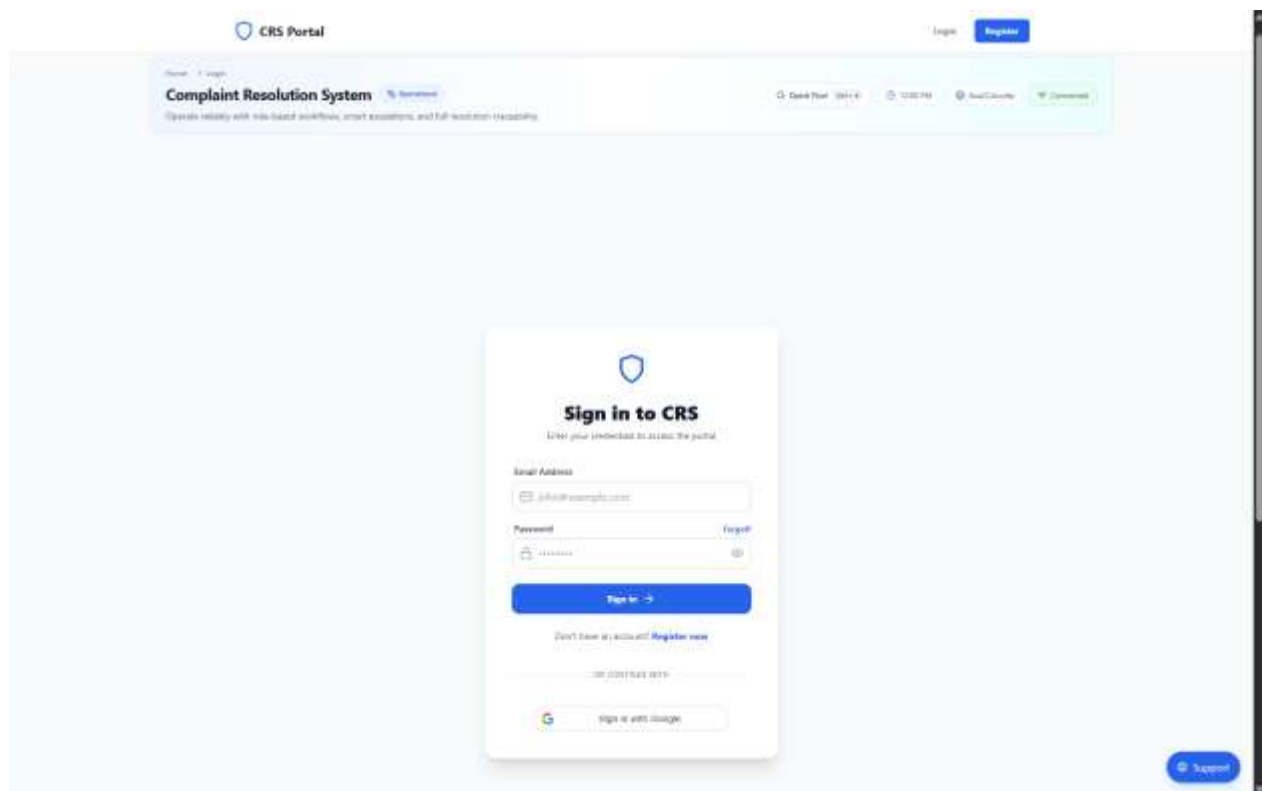


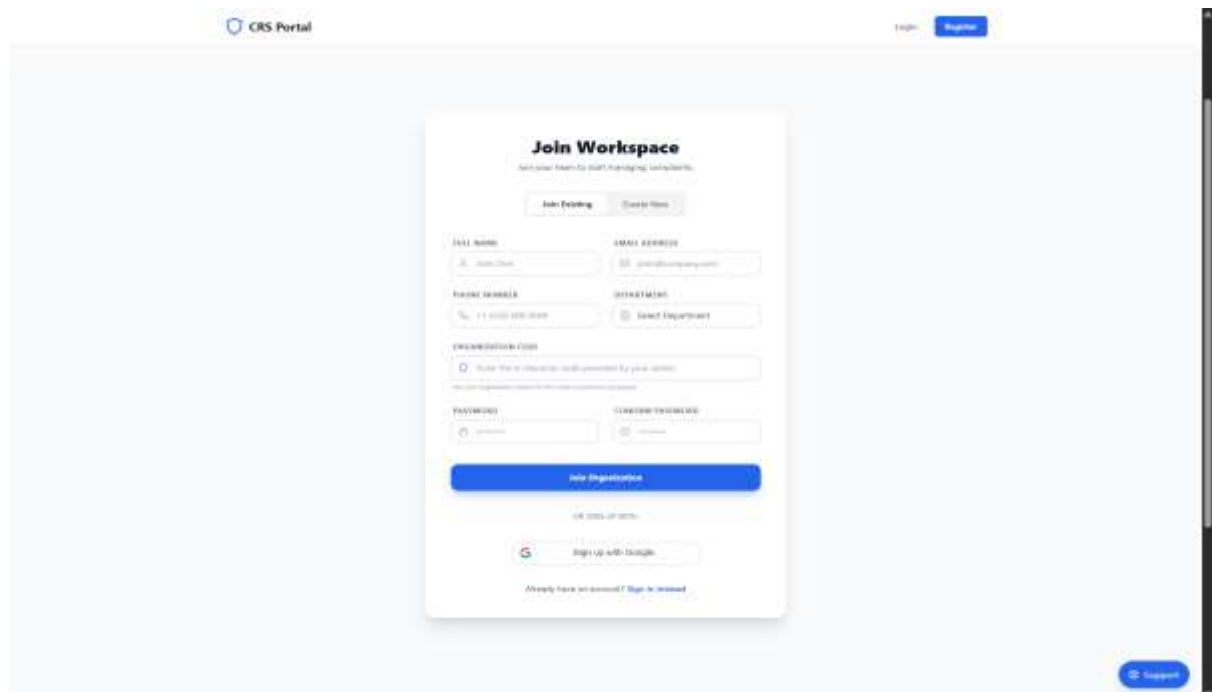
Fig 6.2 shows the Login Page

6.3 REGISTER PAGE

The Register Page allows new users to create accounts with name, email, password, department, and role selection. A real-time password strength meter evaluates complexity and renders a colored progress bar. Google onboarding supports first-time sign-in via Google credentials with an additional profile completion step before account creation is finalized. Email verification is dispatched on standard registration and must be completed before login is permitted.

Backend Integration:

- POST /api/auth/register — Validates email uniqueness, hashes password, creates unverified account, dispatches verification email.
- GET /api/auth/verify-email?token= — Activates account on valid token.
- POST /api/auth/google/onboarding — Completes Google-authenticated first-time account setup.
- GET /api/auth/check-email?email= — Real-time email availability check during form input.



The screenshot displays the 'Join Workspace' registration page. At the top, there is a 'CRS Portal' logo and a 'Sign In' button. The main form is titled 'Join Workspace' and includes a sub-header 'Join our team to start managing workloads.' Below this, there are two tabs: 'Join Existing' and 'Create New'. The form fields are organized into two columns: 'Full Name' and 'Email Address' in the first row; 'Phone Number' and 'Department' in the second row. There is a 'PROVISIONING CODE' section with a dropdown menu. Below that are 'PASSWORD' and 'CONFIRM PASSWORD' fields with a strength indicator. A large blue 'Join Registration' button is positioned below the password fields. At the bottom, there is a 'Sign Up with Google' button and a link for 'Already have an account? Sign In instead.' A 'Support' button is located in the bottom right corner.

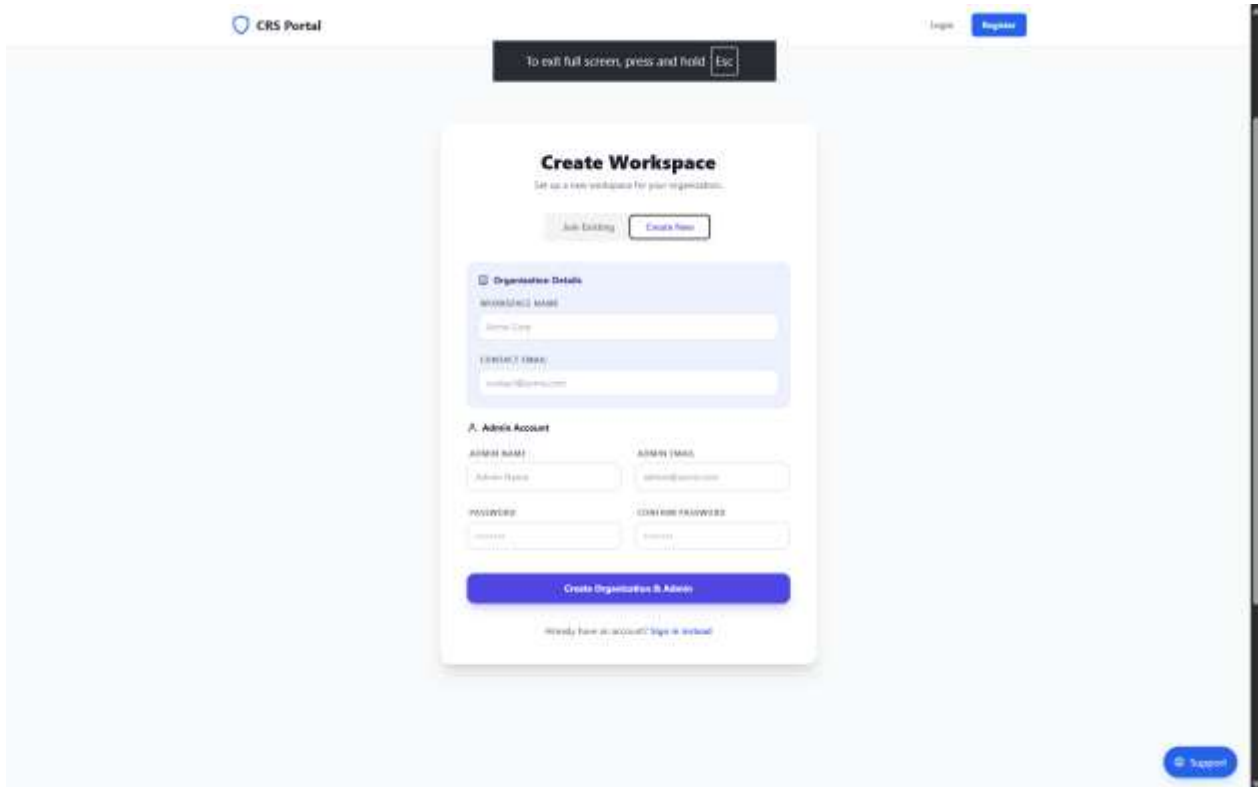


Fig 6.3 shows the Register Page

6.4 DASHBOARD PAGE

The Dashboard is a unified route component that dynamically renders the appropriate dashboard view based on the authenticated user's role. The User Dashboard shows personal complaint statistics and recent complaint cards. The Staff Dashboard displays active assignments, overdue task urgency panel, current capacity indicators, and SLA countdown cues. The Admin Dashboard presents system-wide KPI summary cards, complaint volume bar chart, status distribution pie chart, escalated complaint alerts, and quick-action shortcuts. All dashboards use useEffect with polling for live metric updates.

Backend Integration:

- GET /api/complaints/my-stats — User complaint summary statistics.
- GET /api/staff/dashboard — Staff workload and assignment metrics.
- GET /api/admin/dashboard — System-wide KPI and analytics data.

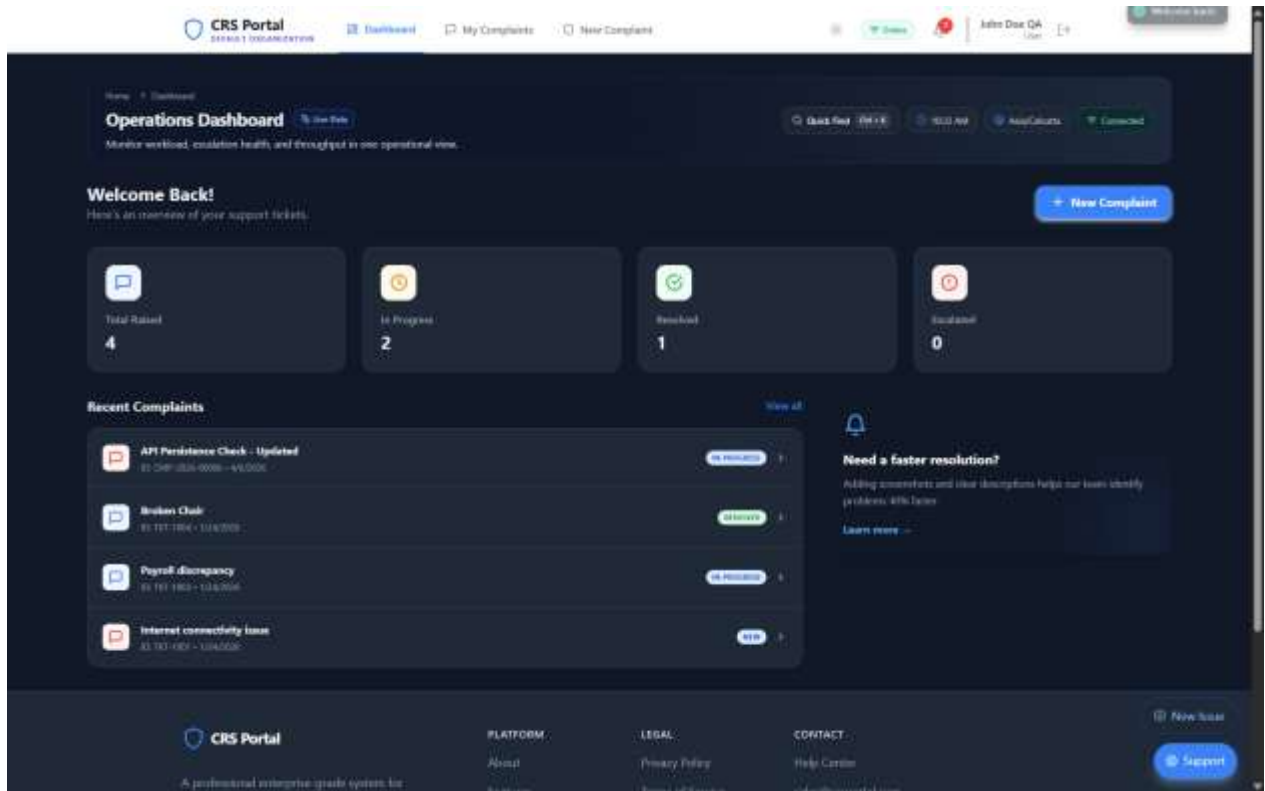


Fig 6.4 shows the Dashboard Page

6.5 COMPLAINT SUBMISSION PAGE

The Complaint Submission Page provides a structured form for users to register a new complaint. `useState` manages the form fields (title, description, categoryId, priority), the attachments array for uploaded files, `validationErrors`, and `isSubmitting` state. The category dropdown is populated from a backend API call on mount. Priority is selected through a visual color-coded button group — Low (grey), Medium (blue), High (orange), Critical (red). File attachments are validated for MIME type and size before upload to Cloudinary. On successful submission a unique complaint reference ID and SLA deadline are displayed in a confirmation screen.

Backend Integration:

- `GET /api/categories` — Fetches active categories for the dropdown.

- POST /api/complaints — Creates complaint, computes SLA deadline, triggers submission notification, returns reference ID.
- POST /api/complaints/:id/attachments — Uploads files to Cloudinary and stores URLs with the complaint record.

The screenshot shows a web application interface for submitting a complaint. At the top, there's a navigation bar with 'CRS Portal' and 'SARAJIT ORGANIZATION'. Below it, a 'Complaint Workspace' header states 'Capture complete issue context, track lifecycle, and keep every stakeholder informed.' The main form is titled 'Submit a New Complaint' with a subtext 'Provide details about your issue and we'll assign it to the right team.' The form contains several input fields: 'Category' (dropdown menu showing 'IT Support'), 'Priority' (dropdown menu showing 'Medium - Standard Issue'), 'Subject / Title' (text input with 'Internet outage in block A'), and 'Detailed Description' (text area with 'the internet is not working for the past one week so kindly work through this and solve the problem since so many students are staying here and internet is a mandatory thing here'). Below these is an 'Attachments (Optional)' section with a dashed box for file uploads and a list of uploaded files, including 'Generated_image_forITSupport.png'. A large blue 'Submit Complaint' button is at the bottom. On the right side, there are links for 'New Issue' and 'Support'.

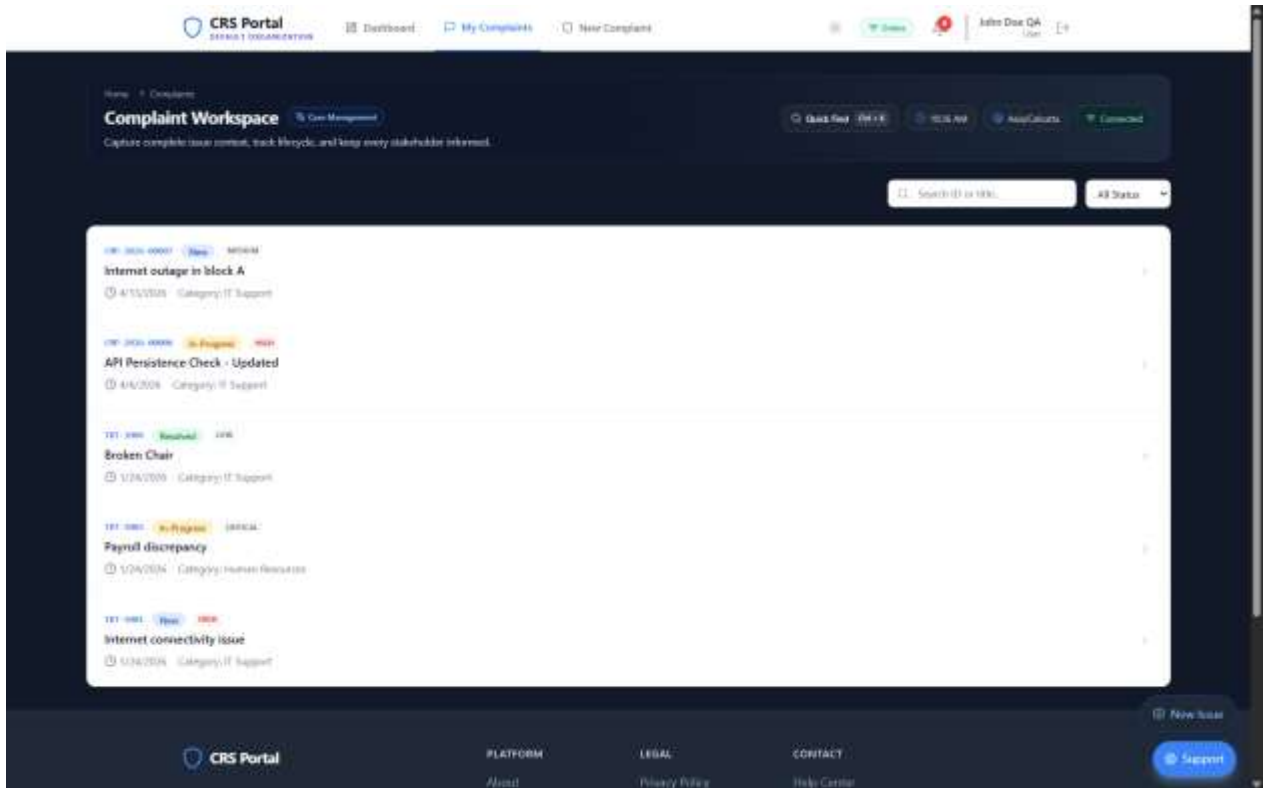
Fig 6.5 shows the Complaint Submission Page

6.6 COMPLAINT LIST AND TRACKING PAGE

The Complaint List Page displays all complaints for the authenticated user with full lifecycle visibility. `useState` manages complaints, `filteredComplaints`, `searchQuery`, filters (status, category, priority, date range), and `selectedComplaint` for the detail modal. A reusable `StatusBadge` component renders color-coded pills per status. The complaint detail modal shows the full status history timeline, assigned staff name, SLA deadline countdown, escalation history, resolution notes, and downloadable attachment links.

Backend Integration:

- GET /api/complaints/my-complaints — Fetches paginated, filtered complaint list for the user.
- GET /api/complaints/:id — Returns full complaint detail with history arrays and attachment URLs.
- GET /api/complaints/:id/timeline — Returns the full chronological timeline for the detail view.
- PATCH /api/complaints/:id/cancel — Cancels an open complaint owned by the user.



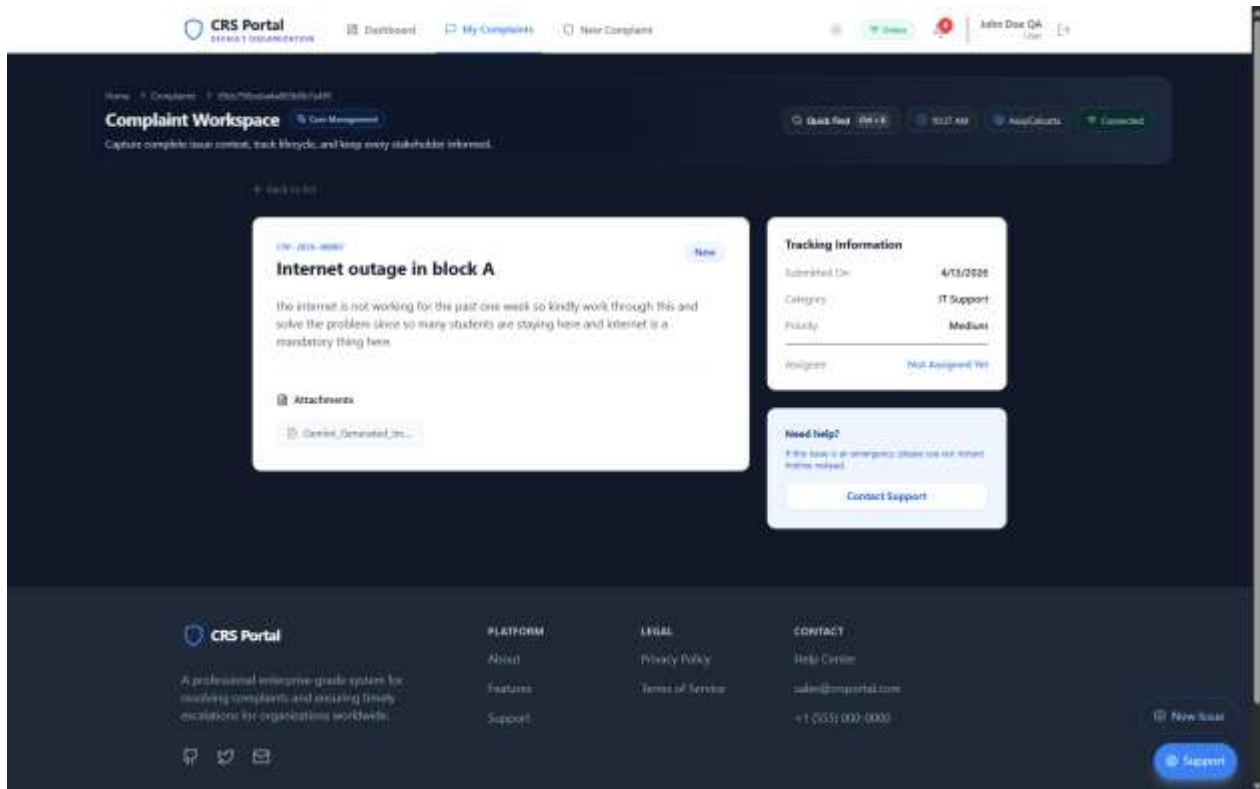


Fig 6.6 shows the Complaint List and Tracking Page

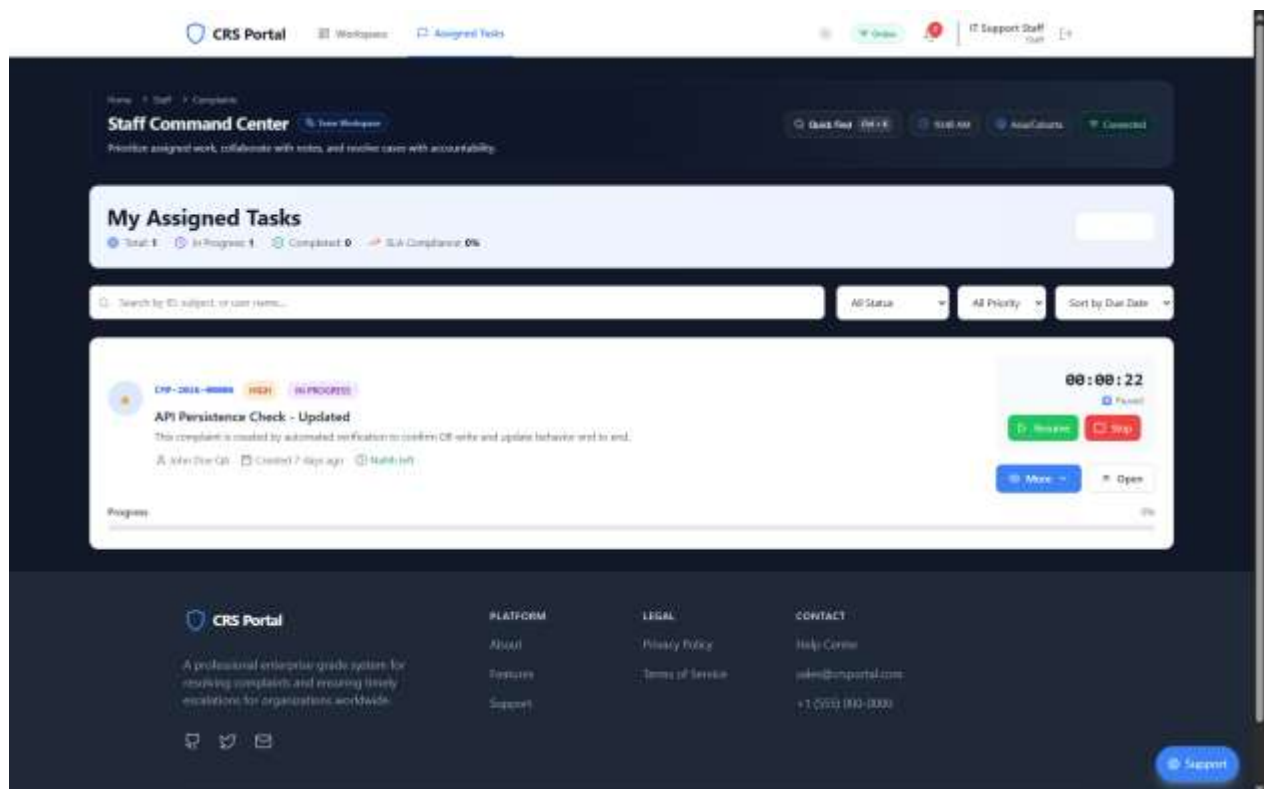
6.7 STAFF RESOLUTION PAGE

The Staff Complaint Workspace provides staff members with a full-screen resolution environment for handling assigned complaints. It shows complete complaint context — title, description, reporter name, category, priority, SLA countdown (turning red below 1 hour) — alongside a resolution action panel with status transition buttons rendered contextually based on current state. Staff can start and pause work sessions, add internal notes, upload resolution evidence, and apply response templates. A Resolution Copilot side panel displays AI-assisted or pattern-matched suggestions from similar previously resolved complaints.

Backend Integration:

- GET /api/staff/complaints — Fetches the staff member's assigned complaint queue.
- PATCH /api/complaints/:id/status — Updates status with notes, triggers complainant notification.

- POST /api/complaints/:id/resolve — Submits resolution summary, marks resolved, triggers feedback request.
- POST /api/complaints/:id/notes — Adds an internal staff note not visible to the complainant.
- POST /api/staff/complaints/:id/work-session — Starts or pauses a work session with timestamp logging.
- POST /api/complaints/:id/evidence — Uploads resolution evidence files.



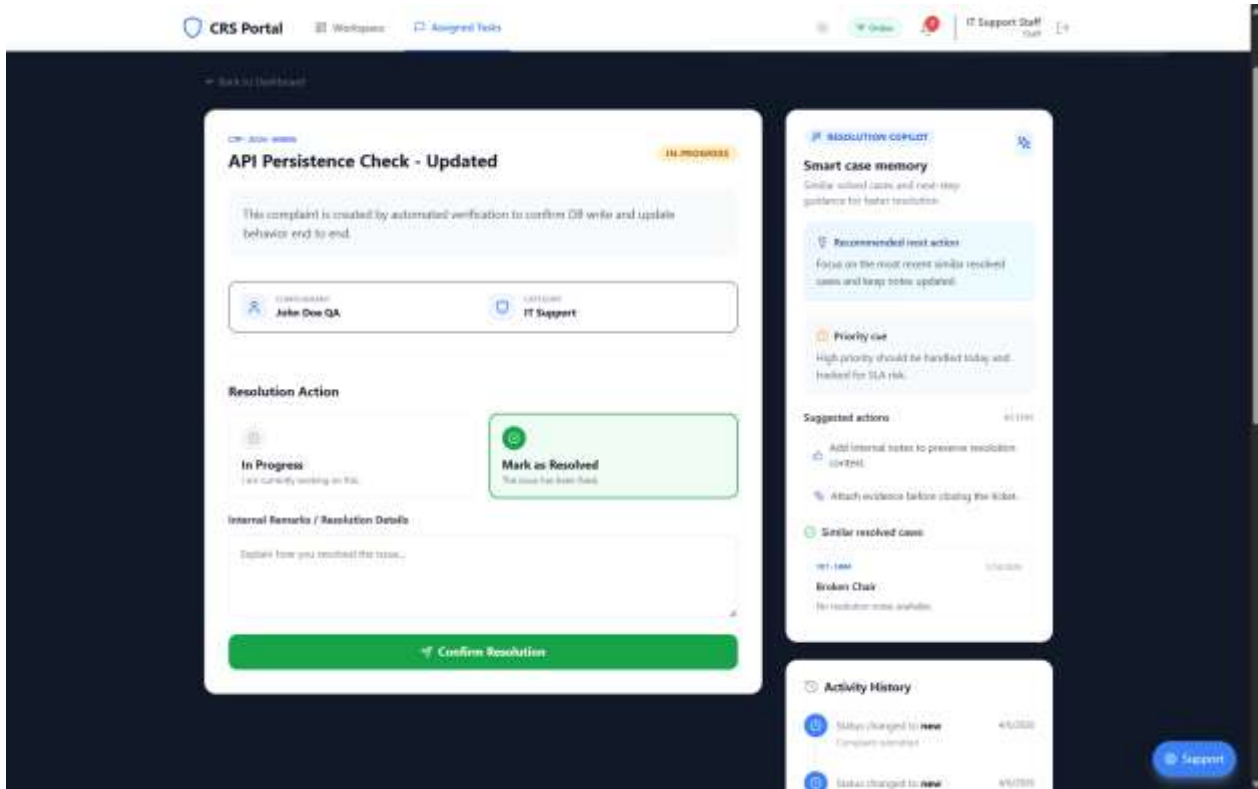


Fig 6.7 shows the Staff Resolution Page

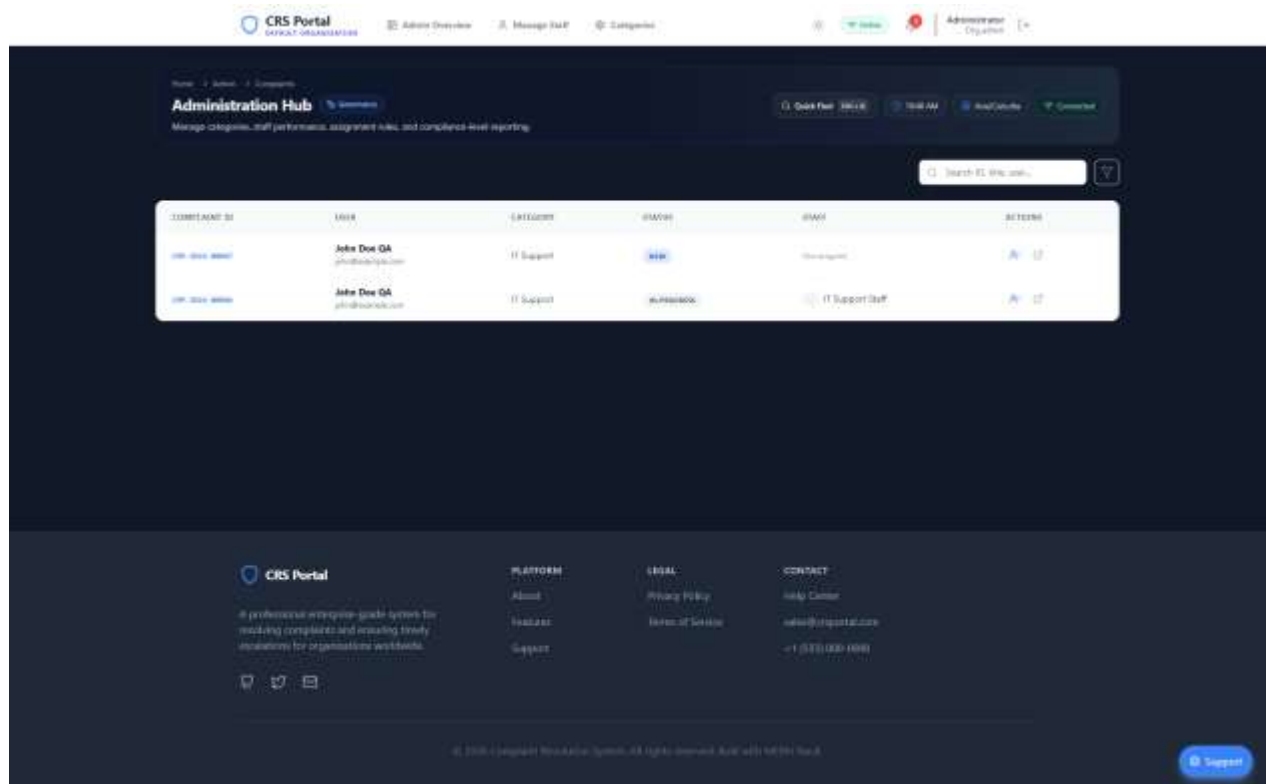
6.8 ASSIGNMENT MANAGEMENT PAGE

The Assignment Management Page enables administrators to assign unassigned complaints to staff with full workload visibility and to configure automated assignment rules. The complaint management table shows all complaints with assign and reassign action buttons. The assignment panel displays each eligible staff member with their current open assignment count and a workload indicator (green/yellow/red). The Assignment Rules Config panel allows admins to create and manage rules with category conditions, priority thresholds, assignment strategy selection (round-robin, workload-based, performance-based, AI-assisted), and activation toggles, with usage analytics shown per rule.

Backend Integration:

- GET /api/admin/complaints/unassigned — Fetches unassigned complaint queue.
- GET /api/staff/workload — Returns staff members with current assignment counts.

- POST /api/assignment/assign — Assigns complaint using active rules or manual selection.
- POST /api/assignment/reassign — Reassigns complaint to a different staff member.
- GET /api/assignment/rules — Fetches all assignment rules with analytics.
- POST /api/assignment/rules — Creates a new assignment rule.
- PUT /api/assignment/rules/:id — Updates an existing rule.
- DELETE /api/assignment/rules/:id — Deletes a rule.



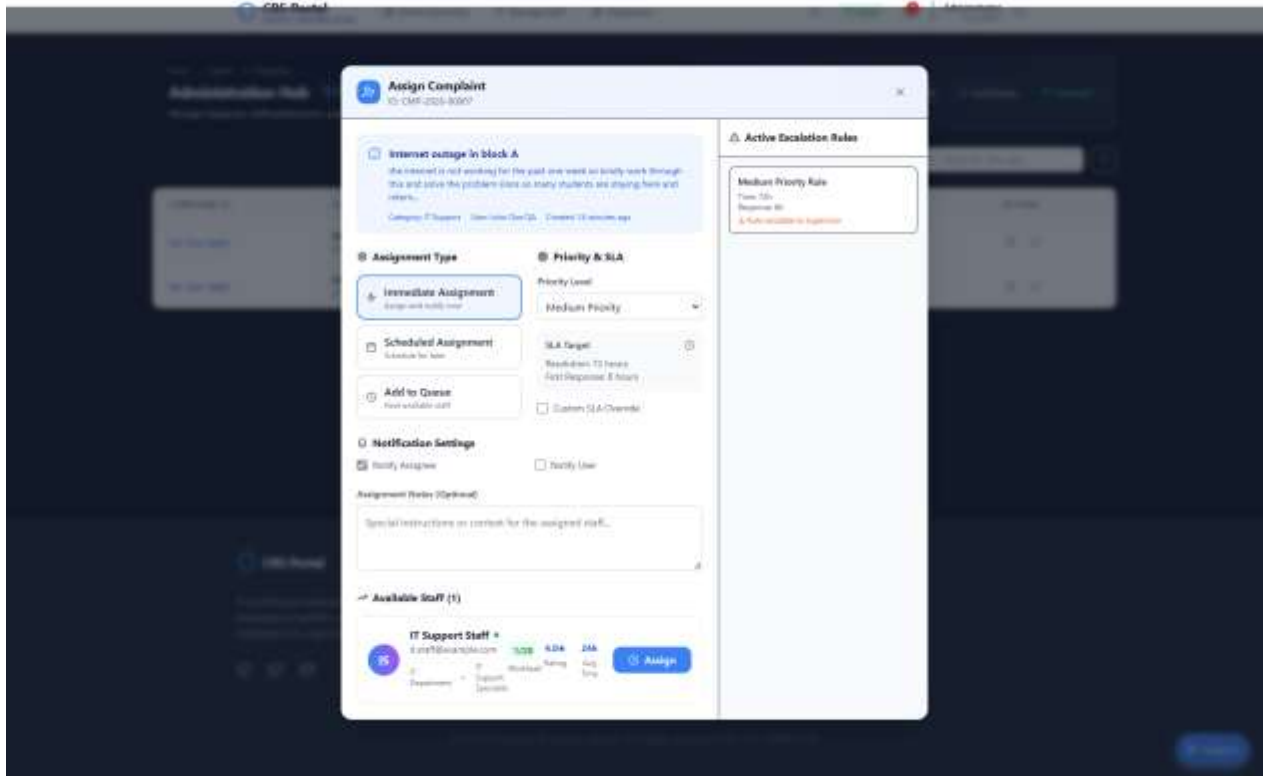


Fig 6.8 shows the Assignment Management Page

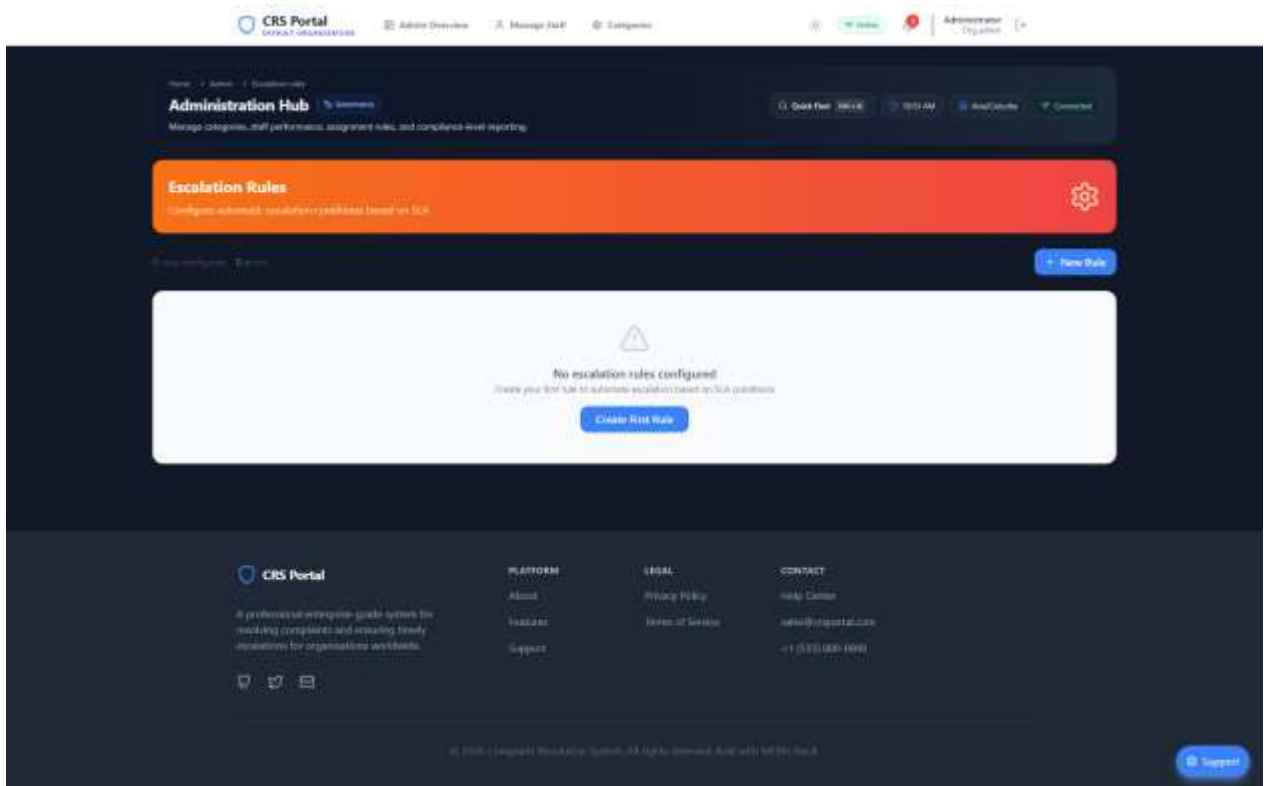
6.9 ESCALATION MANAGEMENT PAGE

The Escalation Management Page allows administrators to configure escalation rules and manually trigger escalation processing for specific complaints. The EscalationRules panel lists all configured rules with their threshold type (time-based, response-based, no-update-based), trigger conditions, configured actions (increase priority, notify supervisor, reassign), and active/inactive status. Admins can create, edit, activate, deactivate, and delete rules. A manual escalation trigger button allows immediate escalation of any active complaint outside of the scheduler cycle.

Backend Integration:

- GET /api/admin/escalation-rules — Fetches all configured escalation rules.
- POST /api/admin/escalation-rules — Creates a new escalation rule.
- PUT /api/admin/escalation-rules/:id — Updates an existing rule.
- PATCH /api/admin/escalation-rules/:id/toggle — Activates or deactivates a rule.

- DELETE /api/admin/escalation-rules/:id — Deletes a rule.
- POST /api/admin/complaints/:id/escalate — Manually triggers escalation for a specific complaint.



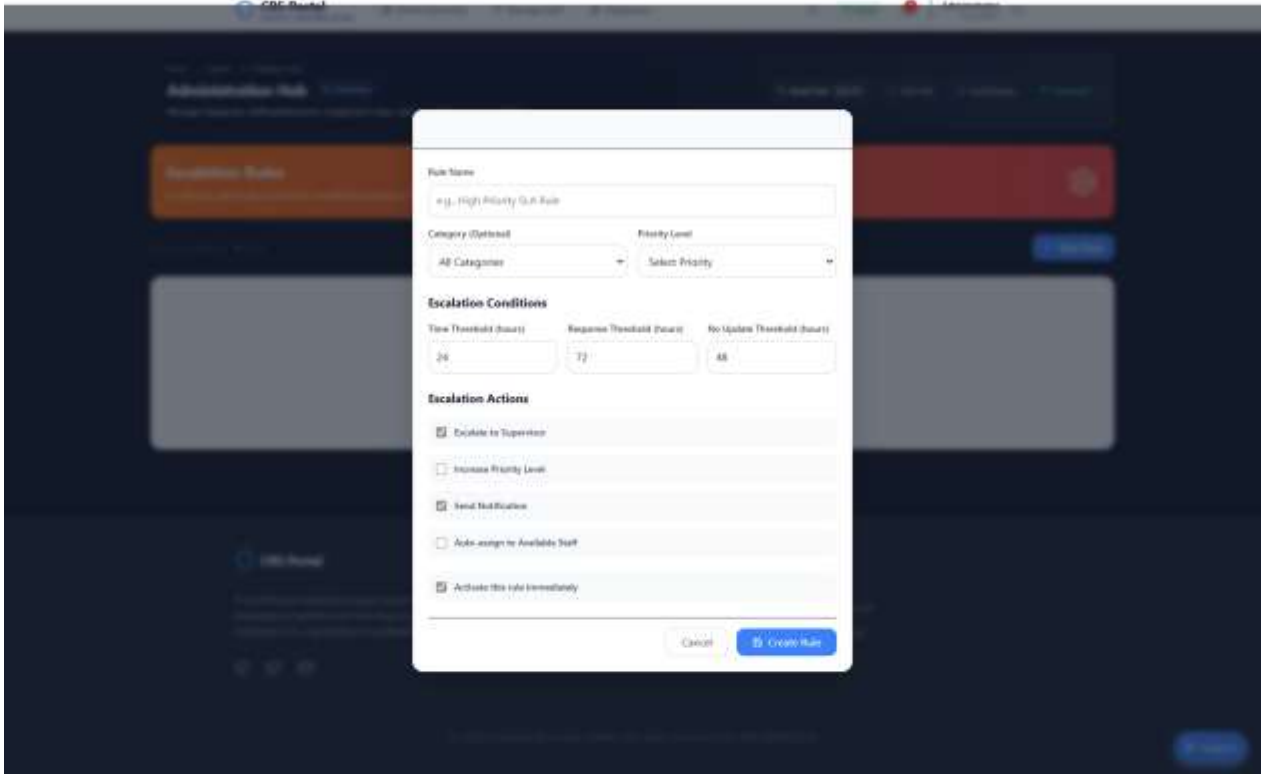


Fig 6.9 shows the Escalation Management Page

6.10 NOTIFICATION PAGE

The Notification Page displays all real-time in-app alerts for the authenticated user covering complaint submissions, assignments, status updates, SLA warnings, escalations, and resolutions. `useState` manages notifications, `unreadCount` for the header badge, `selectedFilter` (All, Unread, Complaints, Escalations, System), and `showNotificationPanel`. Notifications are polled every 30 seconds. Each entry shows a type-specific icon, message text, relative timestamp, unread dot indicator, and action buttons for Mark as Read, Delete, and View Complaint. A Mark All as Read button performs a single bulk-update API call.

Backend Integration:

- `GET /api/users/notifications` — Fetches filtered, paginated notifications for the user.

- PATCH /api/users/notifications/:id/read — Marks a single notification as read.
- PATCH /api/users/notifications/read-all — Bulk-marks all unread as read.
- DELETE /api/users/notifications/:id — Deletes a notification.
- GET /api/admin/notifications/urgent — Fetches high-priority notifications for the admin view.

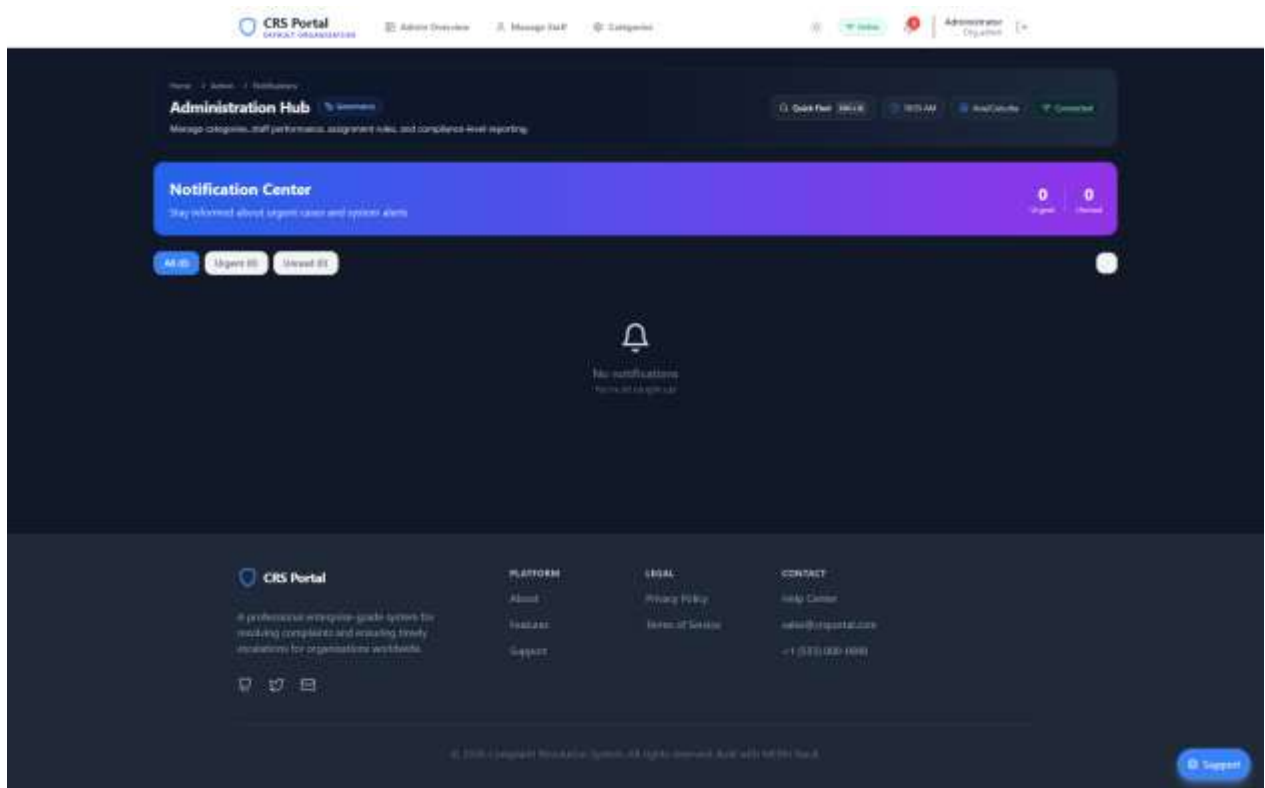


Fig 6.10 shows the Notification Page

6.11 FEEDBACK PAGE

The Feedback Page allows users to submit post-resolution star ratings and qualitative comments for closed complaints, and provides administrators with aggregated analytics for staff performance evaluation. The user-facing view lists all resolved complaints awaiting feedback, each showing the resolution summary and staff name as context. The StarRating component tracks hoveredRating and selectedRating states separately for a clear preview-then-confirm interaction. An anonymous submission toggle is provided. The admin analytics panel shows average rating per staff member and per category with a Recharts trend line.

Backend Integration:

- GET /api/feedback/pending — Fetches resolved complaints awaiting user feedback.
- POST /api/feedback — Submits rating, comment, and anonymous flag; updates complaint to Closed.
- GET /api/feedback/stats — Returns aggregated rating statistics for admin analytics.
- GET /api/feedback/trends — Returns time-series rating trend data for the admin chart.
- GET /api/admin/staff/:id/feedback — Returns feedback records for a specific staff member.

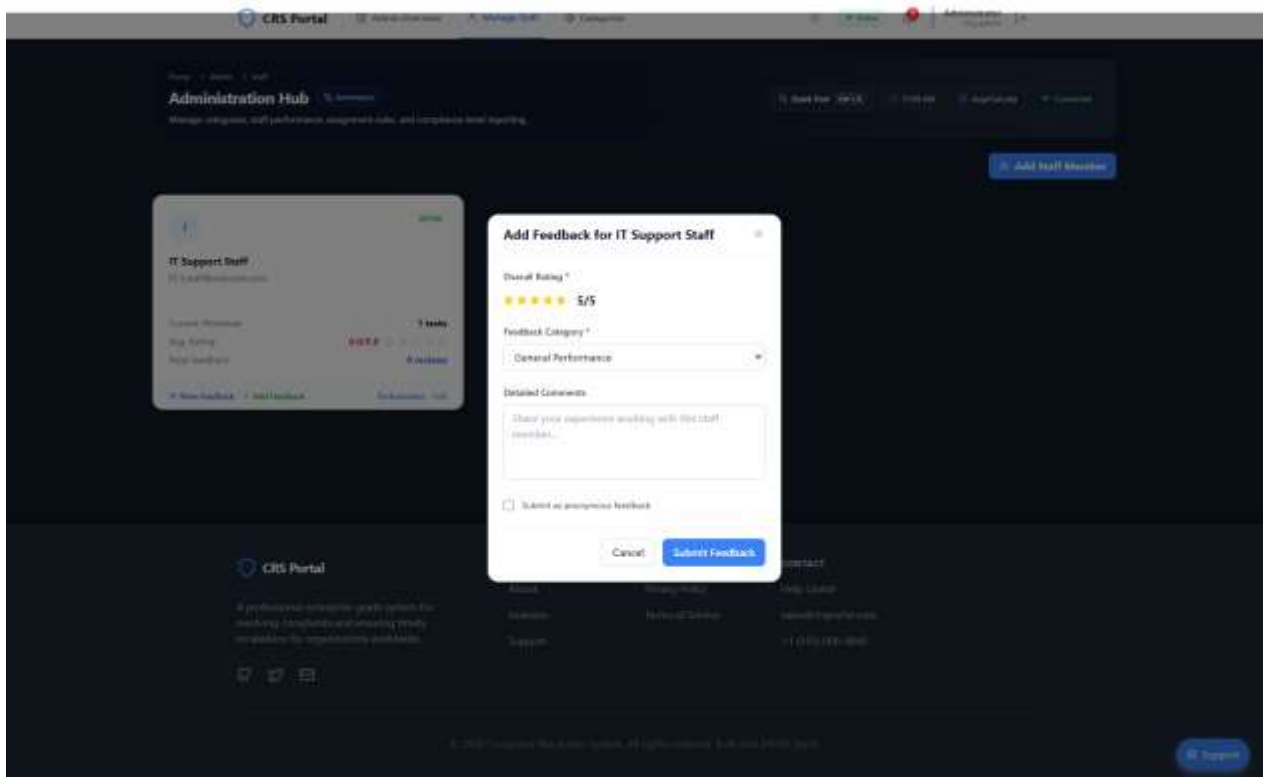


Fig 6.11 shows the Feedback Page

6.12 REPORTS AND ANALYTICS PAGE

The Admin Reports and Analytics Page provides comprehensive data visualization and exportable reports for operational decision-making. `useState` manages `reportType` (Complaint Summary, Staff Performance, Category Analysis, Escalation Report, SLA Compliance), `dateFilters`, and `isGenerating` loading state. `Recharts` renders a line chart for monthly complaint volume trends, a bar chart for category-wise distribution, a pie chart for status breakdown, and horizontal bar charts for staff performance comparison. Export buttons generate downloadable PDF or Excel reports with descriptive filenames.

Backend Integration:

- `GET /api/admin/reports` — Returns aggregated analytics data based on type and date filters.
- `GET /api/admin/reports/export` — Streams downloadable CSV or Excel report for the selected parameters.
- `GET /api/admin/dashboard` — Returns all KPI card metrics for the analytics overview.

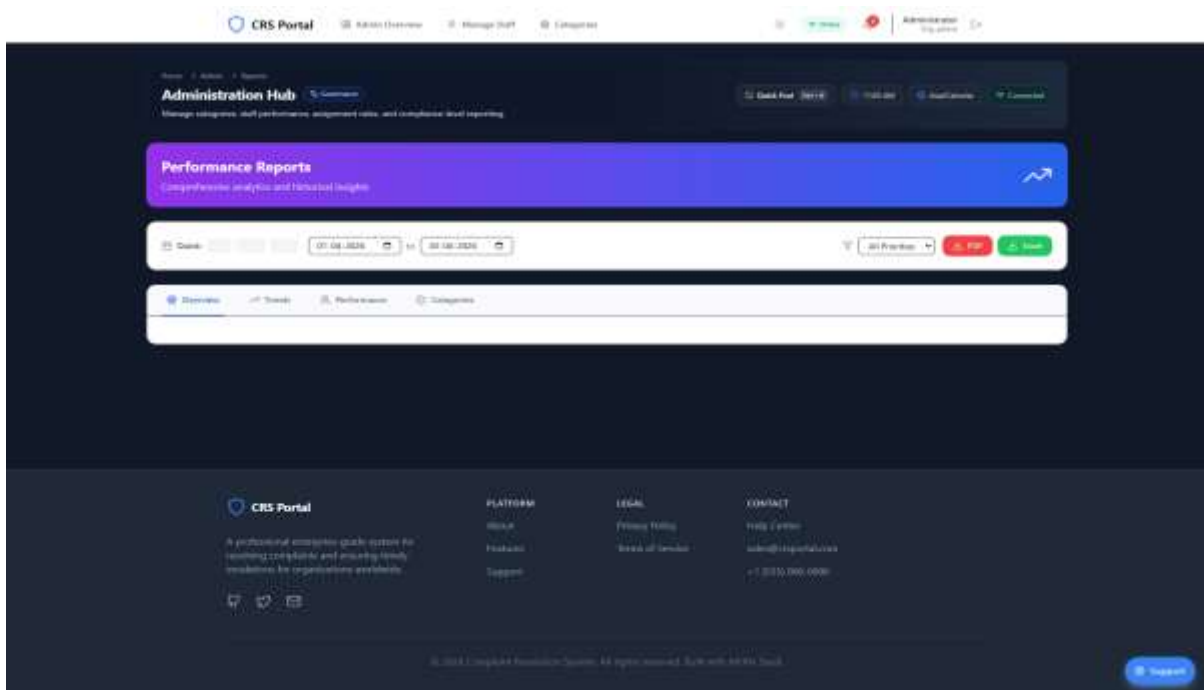


Fig 6.12 shows the Admin Analytics and Reports Page

6.13 CATEGORY MANAGEMENT PAGE

The Category Management Page provides administrators with control over the complaint taxonomy driving routing accuracy and SLA configuration. Each category stores its name, department, description, SLA configuration per priority level, and associated keyword tags used by the AI assignment ranking. A sortable table lists all categories with complaint count badges, SLA hours, and active status toggles. An edit modal allows updating all category fields. A deactivation guard dialog warns the admin of any open complaints in the category before proceeding.

Backend Integration:

- GET /api/categories — Fetches all categories with complaint counts.
- POST /api/admin/categories — Creates a new category with SLA config.
- PUT /api/admin/categories/:id — Updates category fields with audit logging.
- PATCH /api/admin/categories/:id/toggle — Activates or deactivates a category.
- GET /api/admin/categories/stats — Returns complaint volume and SLA stats per category.

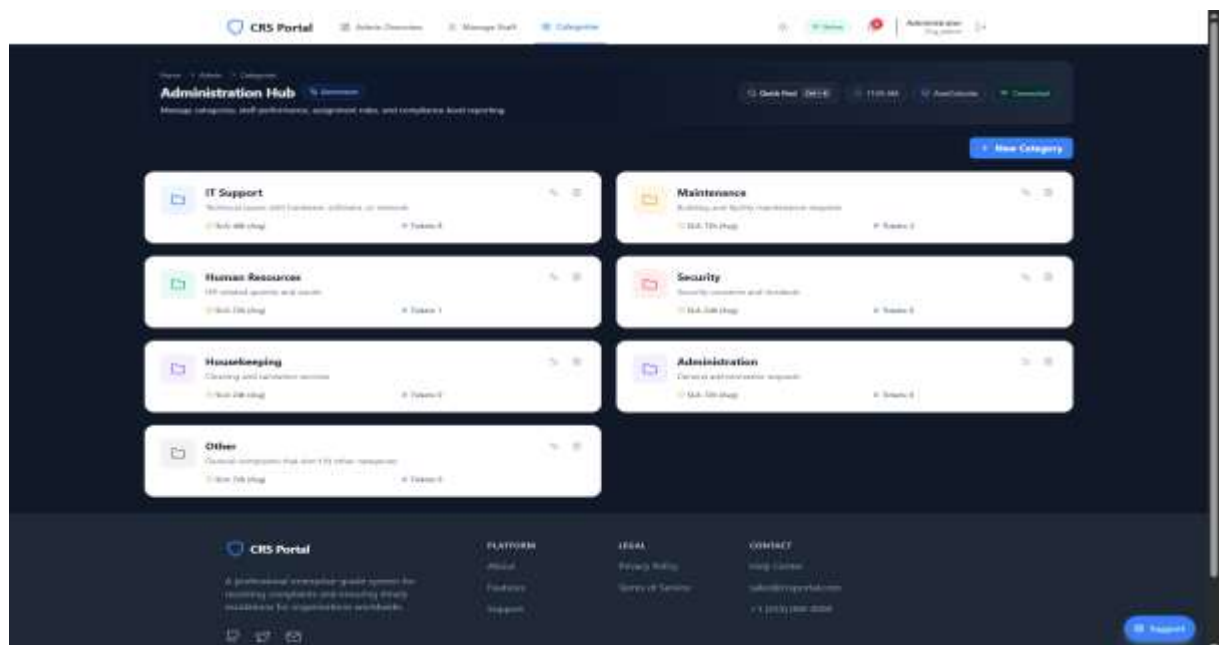


Fig 6.13 shows the Category Management Page

6.14 USER AND STAFF MANAGEMENT PAGE

The User and Staff Management Page gives administrators complete control over all user accounts in the system. `useState` manages users, `filteredUsers`, `searchQuery`, `roleFilter`, `statusFilter`, and `showModal`. A `RoleBadge` component renders color-coded pills per role — Admin (red), Staff (blue), User (grey). The user detail panel shows complaint history, staff resolution metrics, average feedback rating received, last login timestamp, and a mini audit log of recent account-level actions. Staff-specific settings include specialization, availability status, and workload capacity.

Backend Integration:

- `GET /api/admin/users` — Fetches filtered, paginated user list with role and status filters.
- `POST /api/admin/users` — Creates a new user account with temporary password and welcome email.
- `PUT /api/admin/users/:id` — Updates user information and role assignment.
- `PATCH /api/admin/users/:id/status` — Toggles active/inactive with open assignment conflict check.
- `GET /api/admin/staff/performance` — Returns performance metrics for all staff members.

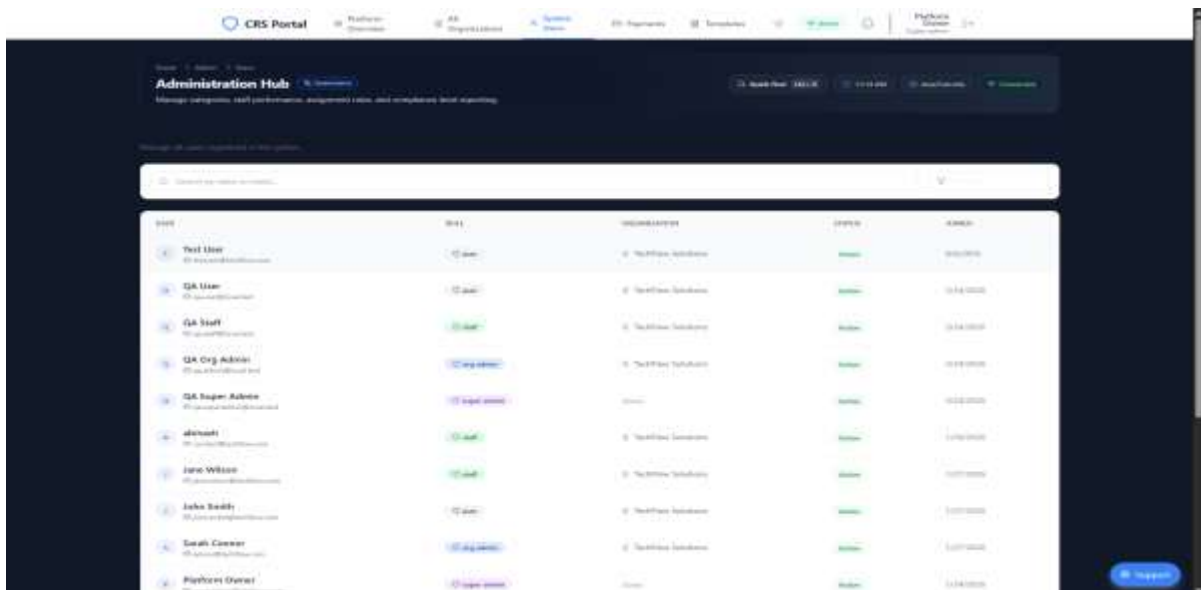


Fig 6.14 shows the User Management Page

6.15 ORGANIZATION MANAGEMENT PAGE

The Organization Management Page provides super-admin level control over all tenant organizations in the multi-organization deployment. Administrators can view all registered organizations with their tenant code, contact email, active status, and creation date. A paginated and searchable table lists all organizations. New organizations can be created through a modal form capturing organization name, tenant code, contact email, and initial admin details.

Backend Integration:

- GET /api/admin/organizations — Fetches paginated list of all organizations.
- POST /api/admin/organizations — Creates a new organization tenant with initial configuration.

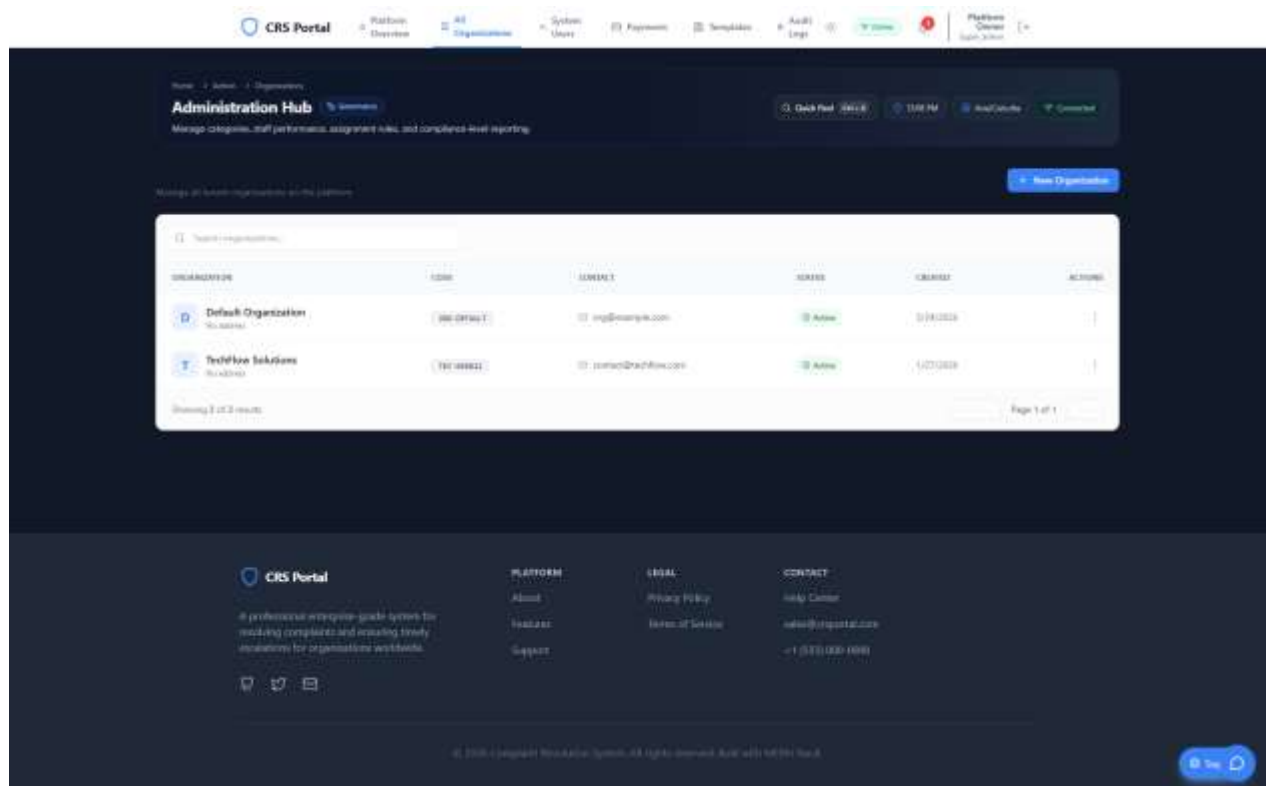


Fig 6.15 shows all organization Management Page

6.16 Payment Center (Upgrade Module)

The Payment Center provides complaint-linked financial workflow management for services that require payment processing. `useState` manages payments, `selectedComplaint` for payment initiation, `paymentHistory`, and `showReceiptModal`. Payment can be initiated against a specific complaint, auto-confirmed in demo mode or processed through the Razorpay test integration.

Successful premium payments trigger a priority boost on the linked complaint. Admins can view the full payment summary, issue refunds, and access transaction audit trails. Staff can view complaint-linked payment acknowledgement status.

Backend Integration:

- `POST /api/payments/initiate` — Creates a payment record linked to a complaint, initiates Razorpay order.
- `POST /api/payments/confirm` — Confirms payment and updates complaint priority if premium.
- `GET /api/payments/my-payments` — Fetches payment history for the authenticated user.
- `GET /api/payments/complaint/:id/status` — Returns payment status for a specific complaint.
- `GET /api/payments/:id/receipt` — Returns receipt data for a completed payment.
- `GET /api/admin/payments/summary` — Returns admin-level payment summary and statistics.
- `POST /api/admin/payments/:id/refund` — Processes a refund for a completed payment.

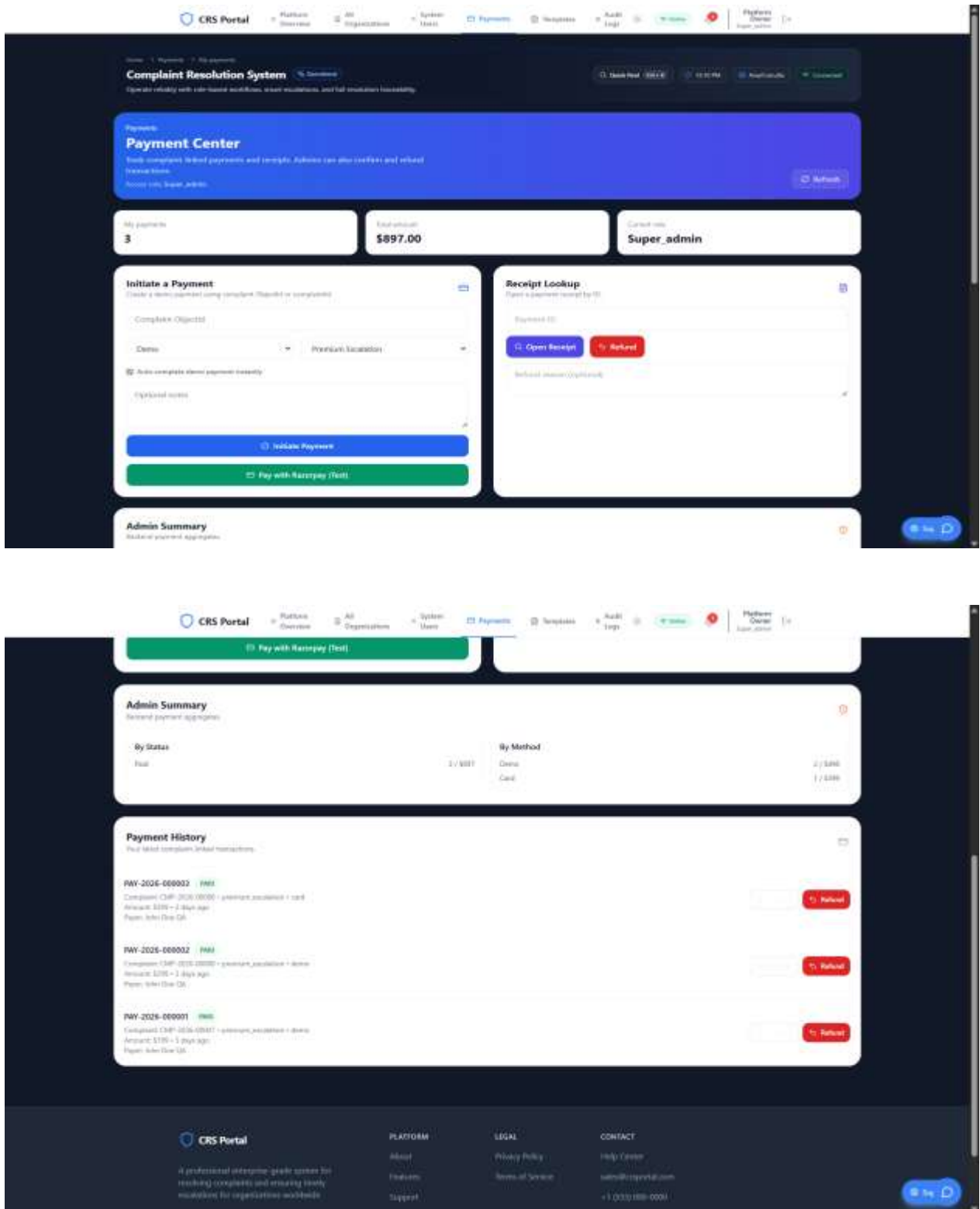


Fig 6.16 shows all Payment Page

6.17 Template Library (Upgrade Module)

The Response Template Library enables staff and administrators to create, manage, and reuse standardized response templates for common complaint scenarios, reducing repetitive manual drafting and improving communication consistency. `useState` manages templates, `selectedTemplate` for editing, `showApplyModal`, and `applyMode` (internal-note or resolution). Templates have a title, body content, audience scope (staff, admin, or all), type classification, and a usage counter that increments on each apply action. Templates can be archived when no longer needed. The apply flow allows injecting a template into a complaint as an internal note or as the resolution summary, with an optional status transition triggered on apply.

Backend Integration:

- GET `/api/templates` — Fetches all active templates filtered by audience scope.
- POST `/api/templates` — Creates a new template.
- PUT `/api/templates/:id` — Updates template content and metadata.
- PATCH `/api/templates/:id/archive` — Archives a template.
- POST `/api/templates/:id/apply` — Applies template to a complaint in note or resolution mode.

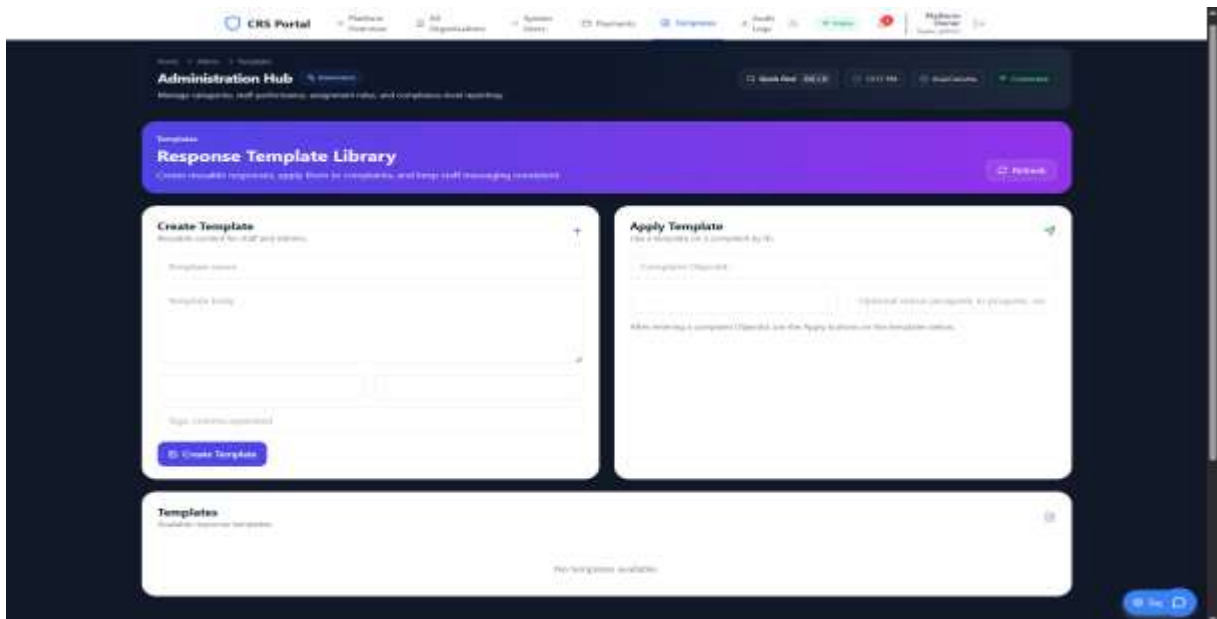


Fig 6.16 shows Template Page

6.18 Audit Log Panel (Upgrade Module)

The Audit Log Panel provides governance-level traceability for all critical system operations. Every log entry captures the actor's name and role, the action type (CREATE, UPDATE, ASSIGN, ESCALATE, DELETE, LOGIN, EXPORT), the target entity type and ID, the HTTP status code, the originating IP address, and a precise UTC timestamp. Entries are displayed in a paginated table with color-coded action type badges. Filter controls allow filtering by action type, role, date range, and status code. An export button downloads the filtered log as a CSV for external compliance submissions.

Backend Integration:

- GET /api/admin/audit-log — Fetches paginated, filtered audit log entries (admin only).
- Query parameters support action, role, startDate, endDate, statusCode, and page filtering.

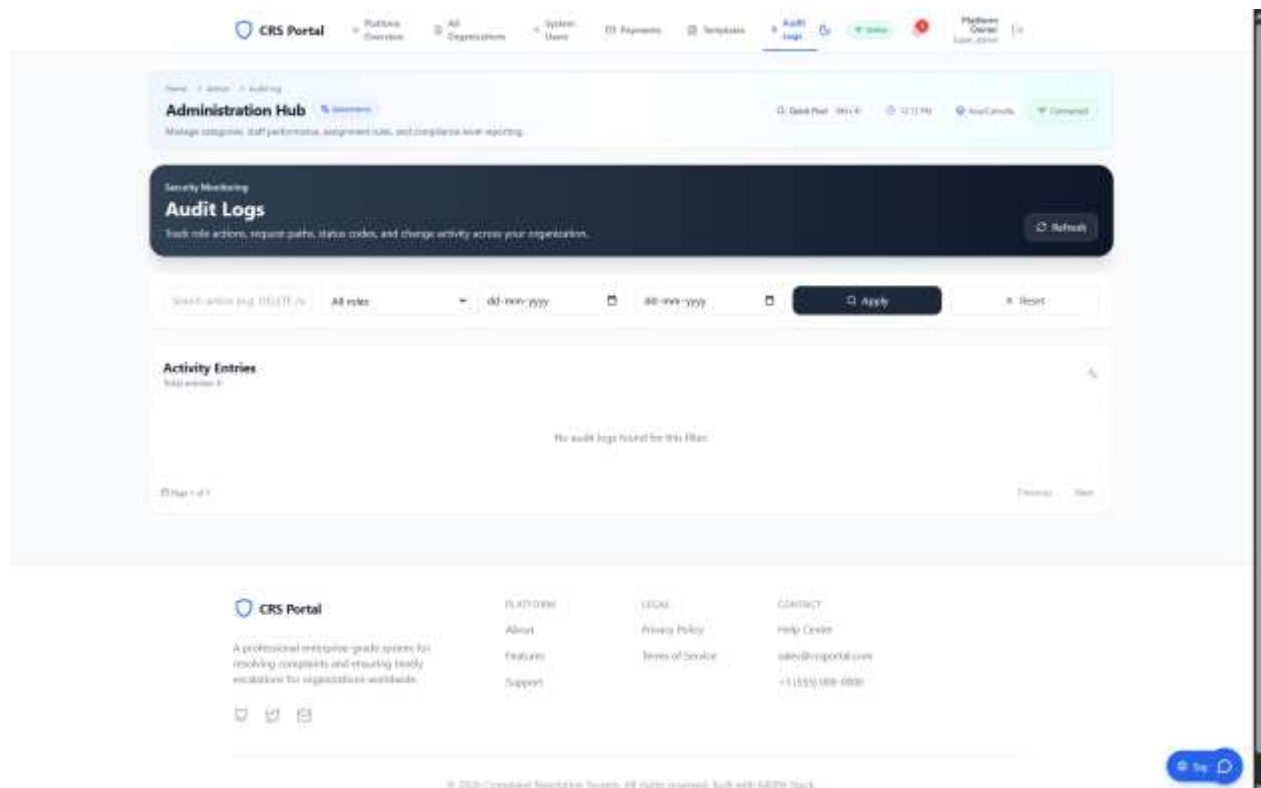


Fig 6.17 shows Audit log Page

6.19 Support Chatbot (Upgrade Module)

The Support Chatbot provides in-app guided assistance for common workflow questions through a floating chat widget embedded in the authenticated frontend layout. The chatbot uses a rule-based intent matching engine that identifies the user's intent from their message and returns a structured guidance response. Supported intents include complaint status enquiry, payment process guidance, login and account help, attachment upload instructions, and escalation status queries. Responses are role-aware — staff users receive workflow-specific guidance while regular users receive complainant-specific instructions. The chat widget opens as a floating panel in the bottom-right corner and maintains conversation history within the session.

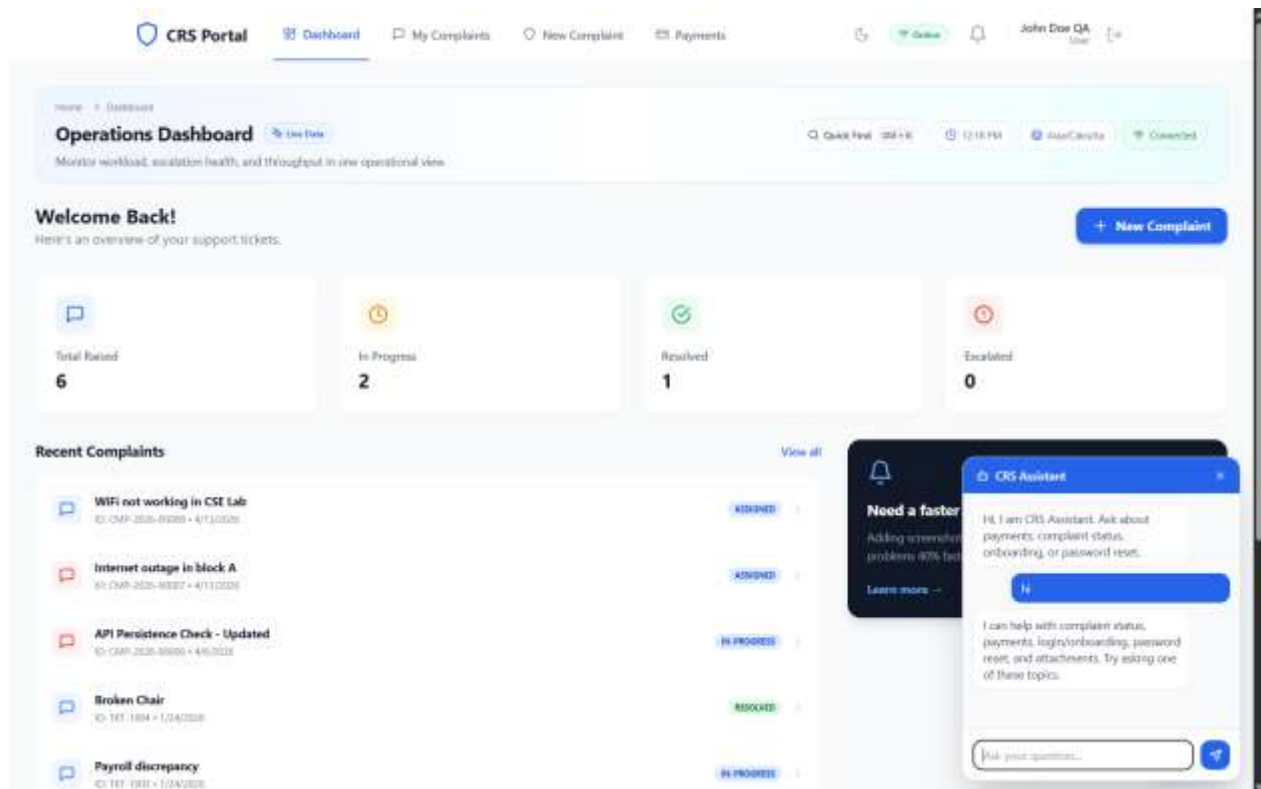


Fig 6.18 shows Chatbot Page

CHAPTER 7

RESULTS ANALYSIS AND DISCUSSION

7.1 Testing Environment

Component	Technology Used
Frontend	React 18, Vite, Tailwind CSS
Backend	Node.js (v18+), Express.js
Database	MongoDB with Mongoose ODM
File Storage	Cloudinary
Email Service	Nodemailer SMTP
Scheduler	node-cron
Payment Integration	Razorpay (test mode)
API Documentation	Swagger UI
Backend Testing	Jest, Supertest
Frontend Testing	Vitest
Containerization	Docker Compose

7.2 Functional Verification Results

Table 7.1 — Functional Test Results

Module	Test Scenario	Expected Result	Observed Result	Status
Authentication	Login with valid credentials	JWT issued, role-based redirect	Token issued, redirected to correct dashboard	Pass
Google Onboarding	First-time Google sign-in	Profile completion screen shown	Onboarding flow triggered correctly	Pass
Complaint Submission	Create complaint with file attachment	Complaint ID generated, SLA computed, Cloudinary URL stored	All fields created and stored correctly	Pass
SLA Computation	High priority complaint in category with SLA config	SLA deadline = creation time + category SLA hours	Correct deadline computed and stored	Pass
Manual Assignment	Admin assigns complaint to staff	AssignmentLog created, both parties notified	Log created, notifications delivered	Pass
Rule-Based Assignment	Complaint matching active rule	Auto-assigned via configured strategy	Correct staff selected using workload strategy	Pass
Work Session Tracking	Staff starts work session on complaint	Session start time logged in work tracking block	Session logged with timestamp in complaint model	Pass

SLA Warning	Complaint approaching deadline	Warning notification sent before breach	Warning dispatched by scheduler at configured threshold	Pass
Auto Escalation	Complaint past SLA deadline	Status updated to Escalated, history entry added	Escalated by cron job, escalation history updated	Pass
Re-Escalation	Complaint stale in escalated state	Escalation level incremented again	Re-escalation triggered by scheduler correctly	Pass
Manual Escalation	Admin triggers escalation manually	Escalation level incremented, notifications sent	Manual escalation processed and logged	Pass
Payment Initiation	User initiates payment for complaint	Payment record created, Razorpay order generated	Payment initiated with correct complaint reference	Pass
Payment Confirmation	Payment confirmed via demo flow	Status updated to paid, priority boost applied	Complaint priority updated, receipt generated	Pass
Template Apply	Staff applies template to complaint	Template body inserted as note or resolution	Applied correctly in selected mode, usage counter incremented	Pass

Feedback Submission	User rates resolved complaint	Rating stored, complaint status updated to Closed	Feedback created, complaint closed, audit logged	Pass
Audit Log Capture	Admin performs critical action	AuditLog entry created with actor, action, IP, timestamp	Log entry created with all required metadata	Pass

7.3 Performance and Operational Metrics

Table 7.2 — System Performance Comparison

Metric	Before CRS — Manual System	After CRS Deployment	Improvement
Average first response time	24 – 48 hours	2 – 4 hours	~85% reduction
Average complaint resolution time	5 – 7 days	1 – 2 days	~70% reduction
Complaints without status visibility	~100% (no tracking exists)	Near zero	Complete improvement
SLA breach rate	Not measurable — no SLA	Below 10% with scheduler active	Enforced and measurable
Escalation consistency	Manual and unreliable	Fully automated — zero-miss	Consistent enforcement
Response standardization	Zero — fully manual drafting	High — template library in use	Significant improvement
Feedback collection rate	Near zero — no mechanism	Above 80% post-resolution	Dramatic increase

Payment process visibility	None — informal collection	Full lifecycle tracked in system	Fully digitized
Admin decision-making time	Days — manual data collection	Minutes — live dashboard	Dramatic improvement
Audit evidence availability	None	Complete — every action logged	Full governance coverage

7.4 Upgrade Impact Analysis

The payment module adds complaint-level financial state visibility and a controlled premium priority boost flow that was entirely absent before. The template module measurably improves response consistency and reduces staff communication drafting time through reusable structured templates. The enhanced assignment engine reduces manual dispatch dependency through rule-based automation with workload-balanced strategy selection. The escalation enhancements — covering warnings, first escalation, re-escalation, and daily summaries — increase SLA governance reliability significantly beyond a basic cron implementation. The audit log and reporting modules transform administrative oversight from reactive verbal check-ins into proactive, evidence-backed governance.

7.5 Discussion

The final upgraded version has evolved from a functional complaint tracker into a policy-governed complaint operations platform. The strongest technical value additions are the end-to-end process orchestration depth connecting assignment, SLA enforcement, escalation, payment, and feedback closure into a single uninterrupted workflow; the administrative governance instrumentation through audit logging, reporting, and analytics; the defensive security implementation integrated at the middleware level rather than as an afterthought; and the extensible modular architecture that allows future AI triage, expanded notification channels, and native mobile clients to be added over the existing API layer without core restructuring.

7.6 Limitations

- The support chatbot is currently rule-based intent matching and not generative. It does not handle free-form queries outside defined intents.
- Frontend test coverage, while present through Vitest utility tests, is narrower than the depth of backend module coverage.
- The payment flow includes demo and test-mode behavior and requires production gateway hardening with real Razorpay credentials for live billing.
- Native mobile application clients are not yet implemented.

7.7 Future Enhancements

- LLM-assisted complaint triage and automatic summarization using OpenAI API integration already stubbed in the backend.
- Predictive SLA breach forecasting using historical complaint resolution time data and machine learning regression models.
- WhatsApp and SMS notification channels via Twilio for broader user accessibility.
- Native Android and iOS client applications built over the existing REST API layer.
- Advanced RBAC policy editor allowing custom role permission configurations per organization.

APPENDIX

Appendix A — Database Collections

User, Organization, Complaint, Category, AssignmentRule, AssignmentLog, EscalationRule, Payment, ResponseTemplate, Notification, Feedback, StaffFeedback, AuditLog, StaffAvailability

Appendix B — Key API Endpoint Groups

- /api/auth — register, login, Google auth, onboarding, refresh, logout, verify-email, forgot-password, reset-password, me, check-email
- /api/complaints — create, my-complaints, my-stats, detail, timeline, update, cancel, reopen, attachments, notes, evidence, resolve
- /api/staff — dashboard, workload, assigned complaints, status update, work-session, internal notes, evidence, resolve, payment acknowledge
- /api/admin — dashboard, complaints list, assign, reassign, escalate, staff management, performance, analytics, reports, export, import, audit-log, escalation-rules, organizations, categories, users, payment summary, refund
- /api/assignment — rules CRUD, assign, reassign, bulk-assign, analytics, SLA info
- /api/payments — initiate, confirm, my-payments, complaint status, receipt, summary, refund
- /api/templates — list, create, update, archive, apply
- /api/feedback — submit, pending, check, stats, trends, moderation, admin management
- /api/categories — public list, tree, suggest, admin CRUD, stats
- /api/users — profile, update profile, preferences, notifications management
- /api/notifications — fetch, mark-read, mark-all-read, delete
- /api/chatbot — message (protected, role-aware)

Appendix C — Folder Structure

complaint-resolution-system/

```
|— backend/
|   |— config/      (db.js, swagger.js)
|   |— controllers/ (auth, complaint, admin, assignment,
|       escalation, payment, template,
|       feedback, chatbot, category, user)
|   |— middleware/  (auth, roleCheck, rateLimiter,
|       csrf, sanitizer, securityHeaders,
|       auditLogger, errorHandler)
|   |— models/      (User, Complaint, Category,
|       AssignmentRule, AssignmentLog,
|       EscalationRule, Payment,
|       ResponseTemplate, Notification,
|       Feedback, AuditLog, Organization)
|   |— routes/      (authRoutes, complaintRoutes,
|       adminRoutes, assignmentRoutes,
|       paymentRoutes, templateRoutes,
|       feedbackRoutes, chatbotRoutes,
|       staffRoutes, userRoutes,
|       categoryRoutes, notificationRoutes)
|   |— utils/        (escalationScheduler.js, emailService.js,
|       cloudinary.js, tokenUtils.js)
|   |— __tests__ /   (middleware, utils, integration, business-logic)
|   |— app.js
|   |— server.js
|— frontend/
```

```

|   └─ src/
|     └─ pages/
|     └─ components/
|       └─ admin/ (AdminDashboard, ComplaintManagement,
|         └─ AssignmentRulesConfig, EscalationRules,
|           └─ PaymentCenter, TemplateLibrary,
|             └─ AuditLogPanel, ReportingModule,
|               └─ OrganizationManagement)
|         └─ staff/ (StaffDashboard, ComplaintWorkspace,
|           └─ ResolutionCopilot)
|         └─ user/ (UserDashboard, ComplaintList)
|         └─ common/ (SupportChatbot, Notifications,
|           └─ StatusBadge, StarRating)
|       └─ context/ (AuthContext, NotificationContext)
|       └─ hooks/
|       └─ services/ (authService, complaintService,
|         └─ adminService, paymentService,
|           └─ templateService, feedbackService)
└─ docker-compose.yml
└─ .env.example
└─ README.md

```

Appendix D — Updated Complaint Lifecycle Workflow

1. User submits complaint with title, description, category, priority, and file attachments.
2. System generates a unique complaint reference ID, computes the SLA deadline from category/priority config, and dispatches a submission notification.

3. Admin reviews the unassigned complaint queue and assigns it manually, or the assignment rules engine auto-assigns based on configured conditions and strategy.
4. Staff receives an assignment notification, opens the Complaint Workspace, starts a work session, and updates status to In Progress.
5. The escalation scheduler scans every 15 minutes — dispatches an SLA warning notification when the deadline is within the configured warning window.
6. If the deadline passes without resolution, the scheduler marks the complaint as breached, escalates to level one, and notifies the configured escalation target.
7. If the complaint remains stale in the escalated state beyond the re-escalation threshold, the scheduler increments the escalation level again.
8. Staff resolves the complaint using a template response or custom resolution summary, uploads resolution evidence, and marks the complaint as Resolved.
9. If the service requires payment, a payment is initiated and confirmed before or after resolution as configured. Successful premium payment triggers a priority boost.
10. User receives a resolution notification and is prompted to submit a star-rating feedback.
11. User submits feedback — complaint status moves to Closed.
12. Admin reviews the full audit log, analytics dashboard, and exported reports for governance and compliance purposes.

REFERENCES

1. Pressman, R. S., and Maxim, B. R., *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education.
2. Sommerville, I., *Software Engineering*, 10th Edition, Pearson Education.
3. Fielding, R. T., *Architectural Styles and the Design of Network-based Software Architectures*, PhD Dissertation, University of California, Irvine, 2000.
4. Tilkov, S., and Vinoski, S., "Node.js: Using JavaScript to Build High-Performance Network Programs," *IEEE Internet Computing*, 2010.
5. OWASP Foundation, *OWASP Top 10 Web Application Security Risks*, <https://owasp.org/www-project-top-ten/>
6. NIST, *Digital Identity Guidelines*, NIST SP 800-63B, National Institute of Standards and Technology, 2017.
7. MongoDB Inc., *MongoDB Aggregation Pipeline Documentation*, <https://www.mongodb.com/docs/manual/aggregation/>
8. Mongoose ODM Documentation, <https://mongoosejs.com/docs/>
9. Node.js Documentation, <https://nodejs.org/en/docs/>
10. Express.js Documentation, <https://expressjs.com/>
11. React Documentation, <https://react.dev/>
12. JWT RFC 7519, *JSON Web Token Standard*, IETF, <https://datatracker.ietf.org/doc/html/rfc7519>
13. Parasuraman, A., Zeithaml, V., and Berry, L., "SERVQUAL: A Multiple-Item Scale for Measuring Consumer Perceptions of Service Quality," *Journal of Retailing*, 1988.
14. Marques, G., and Bhatt, R., "SLA Enforcement and Escalation in IT Service Desks," *Journal of Service Management*, 2019.
15. HDI Research, *Template-Based Communication in Service Desks*, Help Desk Institute, 2018.
16. Cloudinary Documentation, <https://cloudinary.com/documentation>
17. Nodemailer Documentation, <https://nodemailer.com/about/>
18. Razorpay Developer Documentation, <https://razorpay.com/docs/>
19. Swagger OpenAPI Documentation, <https://swagger.io/docs/>
20. Helmet.js, Express-Rate-Limit, and CSRF Middleware Documentation, <https://helmetjs.github.io/>